SMART GARMENT TRACKING AND PRODUCTION DECISION SUPPORT FOR APPAREL PACKING LINES

INDIVIDUAL THESIS

Pathiraja H.P.M.H.D.S.

IT22627582

Individual component: Real Time Good and Rework Garment Counting at the Quality Checking Workstation Using Detection, Multi Object Tracking and Zone Based Verdict Logic

Group ID: R26 IT 054

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) in Information Technology

Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not include, without acknowledgement, material submitted previously for a degree or diploma at any university or other institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I give the Sri Lanka Institute of Information Technology a non-exclusive licence to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while reserving the right to use the contents in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I am responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Pathiraja H.P.M.H.D.S.

Student ID: IT22627582

Signature:  Date: 10/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Ms. Narmada Gamage		
Co-supervisor: Ms. Buddhima Attanayaka		

ABSTRACT

A large hidden cost in apparel manufacturing is rework, but many quality checking tables continue to record it on paper at the end of the shift. This thesis presents the author's component of the Garment Intelligence platform: a Rework Counter that records all accept and reject decisions of a human inspector in real time and without a button, a tag, or any change to the inspector's work. The inspector examines each garment under an overhead camera and places it on a good pile at the top of the view or a rework pile at the bottom. A YOLOv8 nano detector fine-tuned on one g class finds the garment, ByteTrack keeps its identity, and a zone-based state machine logs Good or Rework the first time a track that was inspected in the middle zone crosses the top or bottom line. Verdicts are stored per session and shown as live counts, pass rate, history, and estimated delay. The detector reached a validation mAPE@0.5 of 0.938, precision 0.895 and recall 0.869. All 51 backend and three front end tests passed with 94 % coverage of the service, and two replays of one workflow video gave identical verdicts. A simulation that drives the real counting code with synthetic trajectories showed that the logic is exact on clean input but sensitive to center jitter near the lines, identity switches during the exit movement and spurious short tracks; a three-frame confirmation removed all jitter errors at the cost of more misses when detections drop out. The work shows that the human decision, rather than a defect classifier, can be the source of truth for automated rework counting, and that the reliability of such a counter is decided by the event logic around the model.

Keywords: rework counting, quality inspection, multi object tracking, YOLOv8, apparel manufacturing

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Ms. Narmada Gamage, and my co supervisor, Ms. Buddhima Attanayaka, for their guidance, encouragement and patience throughout this research. Their questions about what the counter really measures, and how sure we could be of each count, shaped the way this thesis evaluates its own results.

I thank the members of my research group, whose components share the Garment Intelligence platform with mine. The integration of four independently developed features into one desktop application taught me as much about software engineering as the computer vision itself, and the discussions we had about interfaces and isolation are reflected in Chapter 5.

I am grateful to the academic and administrative staff of the Faculty of Computing at the Sri Lanka Institute of Information Technology for the knowledge and facilities provided during my degree.

Finally, I owe a deep debt to my family and friends, whose support and understanding made it possible for me to complete this work.

CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 1
1.3 Problem statement 3
1.4 Research gap 3
1.5 Aim and objectives 4
1.6 Research questions or hypotheses 4
1.7 Scope limitations and assumptions 5
1.8 Individual contribution 5
1.9 Thesis structure 6
2 LITERATURE REVIEW AND RELATED WORK 7
2.1 Review method and scope 7
2.2 Domain and theoretical foundations 7
2.2.1 Quality checking and rework in apparel production 7
2.2.2 Human visual inspection 8
2.2.3 Object detection 8
2.2.4 Multi object tracking 8
2.2.5 Counting events with lines and zones 9
2.2.6 Evaluating systems built around models 9
2.3 Related research 9
2.4 Existing systems products and techniques 10
2.5 Gap synthesis and conceptual framework 11
3 RESEARCH METHODOLOGY 13
3.1 Research design 13
3.2 Data participants or study materials 13
3.2.1 Garment piece detection dataset 13
3.2.2 Workflow video and stored sessions 14
3.2.3 Synthetic trajectories 14
3.2.4 Hardware 14
3.3 Experimental or study procedure 15

3.4 Measures and evaluation metrics 16
3.5 Data analysis 18
3.6 Ethics privacy security and responsible AI 18
3.7 Reproducibility and research artefacts 19
4 PRODUCT AND SOLUTION DESIGN 20
4.1 Requirements 20
4.2 Component architecture within the complete solution 21
4.3 Design alternatives and decisions 22
4.4 Detailed design 23
4.4.1 Frame loop 23
4.4.2 Zone geometry 24
4.4.3 Per track state machine 25
4.4.4 Session model and persistence 26
4.4.5 Session lifecycle 27
4.4.6 Interface design 28
4.5 Standards and professional practice 29
5 IMPLEMENTATION INTEGRATION AND TESTING 30
5.1 Development environment and technology stack 30
5.2 Component implementation 30
5.3 Integration with the other components 32
5.4 Verification and software testing 33
5.4.1 Test strategy and suite 33
5.4.2 Representative test cases 34
5.4.3 Execution, coverage and defects 34
5.5 Deployment and operation 35
6 RESULTS AND VALIDATION 37
6.1 Research results 37
6.1.1 Detector training results 37
6.1.2 Verdict logic under controlled conditions 38
6.1.3 Effect of a confirmation rule 42

6.2 Product and technical results 43
6.3 User or stakeholder validation 44
6.4 Integration results 44
6.5 Summary of findings 45
7 DISCUSSION 46
7.1 Interpretation of findings 46
7.2 Comparison with literature baselines and existing solutions 47
7.3 Research contribution 47
7.4 Product contribution 48
7.5 Limitations and threats to validity 48
7.6 Implications 49
8 COMMERCIALISATION AND SUSTAINABILITY 50
8.1 Value proposition and target users 50
8.2 Market and competitive position 50
8.3 Business or sustainability model 50
8.4 Scalability intellectual property and compliance 51
8.5 Contribution to the Sustainable Development Goals 51
9 CONCLUSION AND FUTURE DIRECTIONS 53
9.1 Conclusion 53
9.2 Achievement of objectives 53
9.3 Individual research and product contributions 54
9.4 Future directions 54
REFERENCES 55
GLOSSARY 58
APPENDICES 59
Appendix A Complete list of automated test cases 59
Appendix B Simulation details and complete results 60

LIST OF FIGURES

Figure 1.1: The Rework Counter and the parts it works with at the quality checking station 2
Figure 2.1: Conceptual framework from video frame to quality information 12
Figure 4.1: Process architecture of the Rework Counter inside the platform 21
Figure 4.2: Zones of the frame with the default line settings..... 24
Figure 4.3: State machine applied to every tracker identifier 25
Figure 4.4: Data model and the two queries that serve the interface 27
Figure 4.5: Interactions during one Rework Counter session 28
Figure 6.1: Validation metrics of the garment piece detector during training 37
Figure 6.2: Training and validation losses of the detector 38
Figure 6.3: Correct verdicts as a function of centre jitter with the default lines..... 39
Figure 6.4: Correct verdicts for different line positions at two jitter levels 40
Figure 6.5: Missed garments under detection dropout and identity switches, and extra records from spurious tracks 41
Figure 6.6: Counting error as a function of the number of confirmation frames k 42
Figure 6.7: Verdicts of the three stored sessions over time 43

LIST OF TABLES

Table 2.1: Summary of related technical studies 10
Table 2.2: Comparison of existing approaches with the proposed component..... 11
Table 3.1: Composition of the garment piece detection dataset 13
Table 3.2: Detector training configuration..... 15
Table 3.3: Conditions of the simulation study 16
Table 4.1: Functional requirements of the component..... 20
Table 4.2: Nonfunctional requirements of the component..... 20
Table 4.3: Main design decisions and the alternatives considered 22
Table 4.4: Parameters of the detection, tracking and verdict logic..... 25
Table 4.5: Service endpoints..... 27
Table 5.1: Technology stack of the component 30
Table 5.2: Structure of the automated test suite..... 33
Table 5.3: Representative test cases and their outcomes 34
Table 5.4: Statement coverage of the service module..... 34
Table 5.5: Defects and limitations found during verification 35
Table 6.1: Validation results of the detector at selected epochs 37
Table 6.2: Effect of centre jitter on the verdict logic (600 garments per level)..... 39
Table 6.3: Effect of detection and tracking imperfections (600 garments per level, jitter 2 %) 41
Table 6.4: Counting error (%) for different confirmation lengths k (600 garments each) 42
Table 6.5: Sessions recorded from replays of the workflow video..... 43
Table 6.6: Weaknesses of manual rework recording and the corresponding features44
Table 8.1: Benefits of the component for each user group 50
Table 8.2: Indicative cost structure for one quality checking station 51
Table 9.1: Achievement of the research objectives 53
Table A.1: Automated test cases and outcomes..... 59
Table B.1: Complete results of the simulation study (600 garments per row) 60

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
AMP	Automatic Mixed Precision
AP	Average Precision
API	Application Programming Interface
COCO	Common Objects in Context (dataset)
CPU	Central Processing Unit
CSP	Content Security Policy
CSV	Comma Separated Values
DFL	Distribution Focal Loss
FN	False Negative
FP	False Positive
FPY	First Pass Yield
GPU	Graphics Processing Unit
HOTA	Higher Order Tracking Accuracy
HTTP	Hypertext Transfer Protocol
ID	Identifier
IDF1	Identity F1 score
IoU	Intersection over Union
IPC	Inter Process Communication
JPEG	Joint Photographic Experts Group (image format)
JSON	JavaScript Object Notation
mAP	mean Average Precision
MJPEG	Motion JPEG
MOT	Multi Object Tracking
MOTA	Multi Object Tracking Accuracy
OEE	Overall Equipment Effectiveness
QC	Quality Checking (Quality Control)
REST	Representational State Transfer
ROI	Region of Interest
SDG	Sustainable Development Goal
SQL	Structured Query Language
TP	True Positive
UI	User Interface
WAL	Write Ahead Logging
YOLO	You Only Look Once

1 INTRODUCTION

1.1 Overall project context

Apparel manufacturing is one of the most labor-intensive industries in Sri Lanka and is one of its leading export earners [1]. A garment goes through cutting, sewing, checking, and packing before it is shipped, and at each stage the factory needs to know two things: how many pieces have been completed, and how many of them are fit to go forward. Linking physical work to timely digital information of this kind is a central idea of Industry 4.0 [2], but in many plants the information is still produced by hand, on tally sheets that are totalled long after the work they describe.

The group research project responds to this situation with the Garment Intelligence platform, a single desktop application that brings together four independently developed research components. Each component serves a different workstation: the Rework Counter at the quality checking table, Production Monitoring in the sewing area, Garment Style and Colour Detection at the packing table, and Garment Size Detection for measurement and size verification. The four components share one Electron shell, one Python environment and one installer, but each keeps its own frontend, backend, trained model and database, and the backends never call each other. An operator chooses a workstation card on the landing screen, the platform starts the service that card needs, and the camera of the previous feature is released before the next one opens.

This thesis describes the Rework Counter. One of the four stations it is the one closest to product quality, because it is where a human inspector decides whether a sewn garment can continue or must go back for repair. That decision has a direct effect on delivery: every reworked garment consumes operator time a second time, and a rising rework rate is often the first sign that an order will be late.

1.2 Individual component context

This thesis covers the author's component: **real time counting of accepted and rework garments at the quality checking workstation, using an overhead camera, a garment piece detector, a multi object tracker and a zone based verdict rule.** The component answers three questions for every garment the inspector handles. Is there a garment in the inspection area? Is it the same garment that was there a moment ago? And when it leaves, which pile did the inspector put it on?

The working assumption behind the design is simple and deliberate. The inspector, not the computer, decides whether a garment is good. Training a model to find sewing defects would need a large set of labelled defect images, which a student project cannot collect, and would put a machine judgement in place of a trained person's. Instead the

component reads the decision the inspector has already made from the direction in which the garment is moved. A garment that is inspected in the middle of the view and then placed above the top line is counted as Good; one placed below the bottom line is counted as Rework. Figure 1.1 shows the component together with the people and parts of the platform it works with.

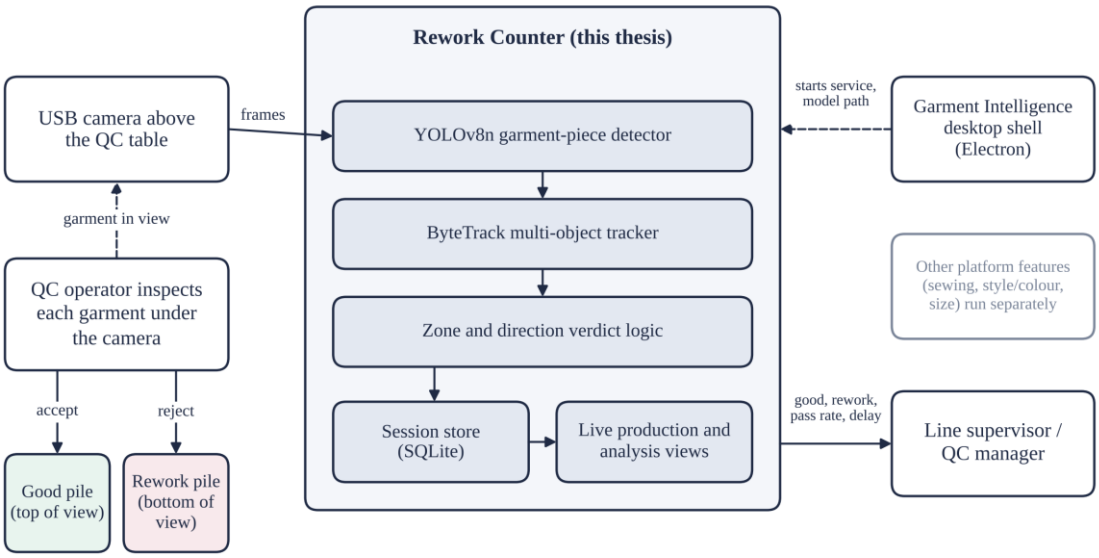


Figure 1.1: The Rework Counter and the parts it works with at the quality checking station

The component spans three layers. A FastAPI service, started by the platform on a private local port, owns the camera or a workflow video, runs a YOLOv8 nano detector [3] with ByteTrack [4], applies the zone logic and writes each verdict to a SQLite database. The Electron main process starts and supervises that service, forwards only an allowed set of requests to it, and relays its video stream. A React interface offers two views: Live production, with the annotated feed, the controls and the live counts, and Analysis and logs, with trend charts, the session log, the session history and CSV export.

The component is designed for the conditions of an ordinary factory. It uses one consumer webcam and the kind of computer already found beside a production line, it must work while inspectors handle garments at normal speed, and it must not add a single step to their routine. These constraints shaped almost every decision described in later chapters, and they also define the main risk: because the counter infers

decisions from motion, any motion that looks like a decision, and any decision that is hidden from the camera, becomes a counting error.

1.3 Problem statement

At a typical quality checking table, accepted and rejected garments are counted by hand, either with a tally sheet or by counting the piles at intervals, and the figures reach the line supervisor and the planning office late. Rework is therefore visible only after it has already cost time. A supervisor who learns at the end of the shift that one in five garments went back for repair has lost the chance to stop the problem at its source during the shift, and the planning office has no basis for revising the delivery date until the next day [5], [6].

Automating the count looks easy. Modern detectors score very well on benchmark data [7], [8], and libraries now offer ready made trackers and line counters [9]. The research problem appears when one asks whether these pieces, put together, turn the ordinary movements of an inspector into a trustworthy record of decisions. The detector only says where a garment is in one frame; the tracker only says which detections probably belong together; neither knows what an accept or a reject looks like. The rule that turns tracks into verdicts carries the whole burden of interpretation, and its behaviour under the imperfect detections and tracks of a real workstation is rarely examined.

The problem is therefore twofold: engineering a component that records accept and reject decisions in real time without changing the inspector's work and without a defect classifier, and understanding how reliable the resulting counts are, which conditions break the verdict logic, and how it could be made more robust.

1.4 Research gap

Computer vision research in textiles has concentrated on automatic fabric and garment defect detection [10], [11], which tries to replace the inspector's judgement rather than record it. Work on human visual inspection documents how often inspectors miss or misjudge defects [12], but treats the inspector as the object of study rather than as the source of a signal. Object counting research has developed line crossing counters for people and vehicles [13], and general tracking research has produced strong multi object trackers evaluated with identity aware metrics [14], [4], [15]. Three gaps follow.

There is little work that uses the direction of a human's handling movement as the label for a quality decision. Counting systems are usually evaluated as a whole on a benchmark video, so the specific contribution of the event logic, as distinct from the detector and tracker, is seldom isolated and tested. And no reviewed system delivers such a counter as an isolated, testable component of a larger factory platform running on ordinary hardware.

1.5 Aim and objectives

The aim is to design, implement and evaluate a real time component that records each accept and reject decision made at an apparel quality checking table from the movement of the garment, stores the decisions per session, and presents them as live quality and delay information. The objectives are:

- **O1.** Train a lightweight single class garment piece detector that can run in real time on the hardware of a quality checking station.
- **O2.** Design a verdict mechanism that infers Good or Rework from the direction in which an inspected garment leaves the inspection area, without any extra action by the inspector.
- **O3.** Persist every verdict with its session and time, and present live counts, pass rate, session history, export and an estimated order delay.
- **O4.** Integrate the component into the Garment Intelligence desktop platform as an isolated feature with its own service, model and data.
- **O5.** Verify the component through automated tests at the interface, logic and persistence levels and through replay of a workflow video.
- **O6.** Characterise the robustness of the verdict logic under realistic imperfections of detection and tracking, and evaluate a refinement.

1.6 Research questions or hypotheses

- **RQ1.** Can a nano scale detector fine tuned on a single garment piece class locate garments on a quality checking table accurately enough to drive a counter?
- **RQ2.** Can the direction of an inspected garment's exit movement, interpreted by a zone based state machine over tracker output, record accept and reject decisions correctly and exactly once?

- **RQ3.** How sensitive is that verdict logic to centre jitter, missed detections, identity switches and spurious short tracks, and which of these matters most?
- **RQ4.** Does requiring a verdict to be confirmed over several frames improve robustness, and at what cost?
- **RQ5.** Can the counter be delivered as an isolated, verifiable feature of a multi component desktop platform?

The working hypothesis was that the detector would not be the limiting factor, because the task needs only one class and a rough position, and that the reliability of the counter would be decided mainly by the event logic and by the quality of tracking near the zone boundaries.

1.7 Scope limitations and assumptions

- One quality checking table observed by one overhead camera, with the good pile at the top of the view and the rework pile at the bottom; the mapping of direction to verdict is fixed in the code.
- One detector class, Garment-Piece-Detection; the component does not identify the type of defect, the style or the size of the garment.
- The inspector's decision is taken as correct. The component measures decisions, not product quality, and cannot detect a garment that was wrongly accepted.
- The estimated delay is the number of rework garments multiplied by an average fix time entered by the user; it is an indicator, not a schedule.
- Evaluation covers the detector's validation metrics, automated tests, two replays of a workflow video and a controlled simulation of the verdict logic. No long trial against an independent manual count at a working line was completed, so no claim is made about field counting accuracy or labour saved.

1.8 Individual contribution

The author prepared the garment piece dataset and trained the detector; designed the zone and direction verdict logic and its per track state machine; built the FastAPI service with its session store, video source handling and live settings; developed the Live production and Analysis views; and adapted the component for the integrated platform, including the isolated service launch, the allow listed requests and the stream

relay. For this thesis the author also wrote an automated test suite of 51 backend tests that exercises the real service with the camera and model replaced by stubs, analysed the stored workflow sessions, and designed the simulation study of the verdict logic reported in Section 6.1. The shared Electron shell, landing screen and release pipeline were developed jointly by the group.

1.9 Thesis structure

Chapter 2 reviews the literature and sets out the conceptual framework. Chapter 3 describes the methodology, Chapter 4 the design and Chapter 5 the implementation, integration and testing. Chapter 6 presents the results and Chapter 7 discusses them. Chapter 8 covers commercialisation and sustainability, and Chapter 9 concludes with future directions.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review was done in three passes. The first covered the quality management background the component serves: inspection, rework, yield and the cost of poor quality. The second covered the techniques the component is built from: object detection, multi object tracking, and the counting of events with virtual lines and zones. The third looked for work close to the problem, namely vision systems in apparel quality control and counting systems that interpret human or object motion, and for the evaluation practices that would show where such systems fail. Sources came mainly from IEEE Xplore, the main computer vision conferences, quality management textbooks and official framework documentation, with preference for peer reviewed and widely cited work.

2.2 Domain and theoretical foundations

2.2.1 Quality checking and rework in apparel production

In apparel factories quality is controlled at several points, from fabric inspection to in line checks at sewing and the final inspection of finished garments [5]. At an end of line quality checking table, each garment is examined against the buyer's specification and either passed forward or returned to the sewing line for repair. Returned garments are rework: they have consumed material and labour once and will consume labour again. Quality management literature treats this as part of the cost of poor quality [6], [16], and lean production lists defects and their correction among the basic forms of waste [17].

Two simple measures summarise a checking table. The first is the share of garments that pass at the first inspection, usually called first pass yield; the second is its complement, the rework or defect rate. Statistical quality control monitors the fraction nonconforming over time, for example with p charts, so that a rise can be acted on before it becomes a delivery problem [18]. Both measures depend entirely on reliable counts of accepted and rejected pieces, collected close to the time the decisions were made. The equipment effectiveness view adds a further reason to care: quality losses reduce effective output in the same way as stoppages and slow running [19].

2.2.2 Human visual inspection

Visual inspection by trained people remains the norm for finished garments. A review of the inspection literature by See [12] shows that human inspectors are flexible and good at judging unfamiliar defects, but that their accuracy varies with fatigue, pace and the clarity of standards, and that recording the result is a separate task that competes with inspection for their attention. For a counter this has two consequences. The decision itself is best left to the inspector, and any recording mechanism should take nothing from the inspector's attention. Reading the decision from a movement the inspector already makes satisfies both.

2.2.3 Object detection

Two stage detectors such as Faster R-CNN [20] first propose regions and then classify them, which makes them accurate but slow. Single stage detectors predict boxes and classes in one pass; the YOLO family introduced by Redmon et al. [7] made real time detection practical and has since evolved through many versions [8]. YOLOv8 [3] uses an anchor free head and offers a nano model of about three million parameters, small enough for CPU or modest GPU inference. Fine tuning from weights pretrained on COCO [21] lets such a model learn a new class from a few hundred or thousand labelled images. Detectors are evaluated by precision, recall and average precision, where a prediction counts as correct if its Intersection over Union with a ground truth box passes a threshold; PASCAL VOC used an IoU of 0.5 [22], COCO averages over thresholds from 0.50 to 0.95 [21], and Padilla et al. [23] survey the subtleties.

2.2.4 Multi object tracking

A counter must know that the garment in one frame is the same garment as in the next. Tracking by detection links detections over time. SORT [14] predicts each track's next position with a Kalman filter [24] and assigns detections to tracks with the Hungarian algorithm [25] on box overlap. DeepSORT [26] adds an appearance descriptor to survive occlusion. ByteTrack [4] showed that low confidence detections, which most trackers discard, are valuable for keeping tracks alive through partial occlusion, and associates them in a second matching stage. BoT-SORT [27] adds camera motion compensation and appearance cues. Tracking quality is measured with identity aware metrics: MOTA from the CLEAR MOT framework [15], identity F1, and HOTA [28],

on benchmarks such as MOT16 [29]. A recurring finding of this literature is that identity switches, where a track's identifier changes for the same object, are a main error source, especially during fast motion and occlusion.

2.2.5 Counting events with lines and zones

Counting people or vehicles that cross a virtual line in video is a long standing problem. Ma and Chan [13] formulated line crossing counting as an integer programming problem over local features, avoiding explicit tracking. With reliable trackers, a simpler approach has become common: a track is counted when the anchor point of its box, usually the centre or the bottom centre, moves from one side of a line to the other. Open source libraries such as Supervision [9] package this as a line zone with in and out counts. The Rework Counter uses the same idea but changes its meaning: it uses two lines and a central inspection zone, requires a track to be seen in the inspection zone before it can be counted, and maps the side on which the track leaves to a quality verdict.

2.2.6 Evaluating systems built around models

Sculley et al. [30] argued that in deployed machine learning systems the model is a small part of the code, and that much of the risk lies in the configuration, data handling and glue logic around it. A counter is a clear example: a detector with excellent average precision can still produce wrong counts if the event logic double counts a jittering box or misses a track whose identity changed at the wrong moment. Benchmarks for detection and tracking do not measure the counting rule, so it has to be tested separately.

2.3 Related research

Vision in apparel quality control. Kumar's survey [10] and the review by Ngan et al. [11] cover automated fabric defect detection on moving material under controlled light. These systems replace the inspector for one specific task, require large defect datasets and controlled imaging, and do not address the counting of decisions at a manual checking table.

Detection and tracking. The YOLO family [7], [3], [8] and trackers such as SORT, DeepSORT, ByteTrack and BoT-SORT [14], [26], [4], [27] provide the building

blocks used here. They are evaluated on benchmarks of pedestrians and vehicles with frame level and identity level metrics [15], [28], [29], not on whether a downstream counting rule produces correct events.

Line crossing counting. Ma and Chan [13] and library implementations [9] count crossings of a line. They count objects, not decisions, and treat every crossing as an event, so an object that hesitates on the line or is handled back and forth can be counted more than once unless extra logic is added.

What the related work leaves open. Taken together, these studies show that garments can be detected and tracked and that crossings can be counted, but none of them interprets a crossing as a quality decision, or isolates and stress tests the event rule that does so. Table 2.1 summarises the studies most relevant to the design and what each leaves open for the quality checking problem.

Table 2.1: Summary of related technical studies

Study	Contribution	Limitation for the quality checking problem
Redmon et al. [7]	Real time single stage detection	Frame level only; no notion of events
Jocher et al. [3]	YOLOv8 with a nano variant and training toolkit	No guidance on counting or tracking rules
Bewley et al. [14]	Kalman filter and Hungarian matching tracker	Identity switches under occlusion and fast motion
Zhang et al. [4]	Associates low confidence detections	Evaluated on pedestrians, not handled objects
Ma and Chan [13]	Line crossing counting	Counts crossings, not decisions; no inspection zone
Kumar [10]; Ngan et al. [11]	Automated fabric defect detection	Replaces the inspector; needs defect data and controlled light
See [12]	Review of human visual inspection	Studies inspectors, does not record their decisions
Sculley et al. [30]	Risks of the code around models	General; not tied to counting

2.4 Existing systems products and techniques

Table 2.2 compares the ways a quality checking table can record its results with the component developed here. Paper tally sheets cost nothing but add a recording task to the inspector, introduce delay and leave no time record. Counting the piles at intervals is quicker but gives no timing at all. Tablet or handheld entry gives timely data but still asks the inspector to press a button for every garment. Barcode or tag scanning links each decision to a specific garment, which is valuable for traceability, but needs tags,

readers and a scanning step. Automated defect inspection replaces the human decision and is far more demanding to build and validate.

Table 2.2: Comparison of existing approaches with the proposed component

Approach	Real time	Extra action by inspector	Per garment time record	Extra hardware or tagging
Paper tally sheet	No	Yes, every garment	No	None
Counting piles at intervals	No	Periodic	No	None
Tablet or handheld entry	Yes	Yes, every garment	Yes	Tablet
Barcode or tag scanning	Yes	Yes, a scan	Yes, per garment ID	Tags, readers
Automated defect inspection [10], [11]	Yes	No	Yes	Controlled imaging, defect dataset
Proposed component (this work)	Yes	No	Yes, per verdict	One USB camera

2.5 Gap synthesis and conceptual framework

The literature offers mature building blocks but no component that uses the inspector's own handling movement as the record of a quality decision, and no evaluation that isolates the event logic on which such a counter depends. The framework adopted here, shown in Figure 2.1, treats the counter as a chain of increasingly specific questions. A frame is reduced to detections, which say where garment pieces are. The tracker turns detections into identities, which say which garment is which. The zone logic arms an identity only after it has been seen in the inspection zone, which says that the garment was actually inspected. The direction in which an armed identity leaves says which pile it went to, and that becomes a verdict record. Only the records are interpreted, as a pass rate and an estimated delay, using standard quality measures.

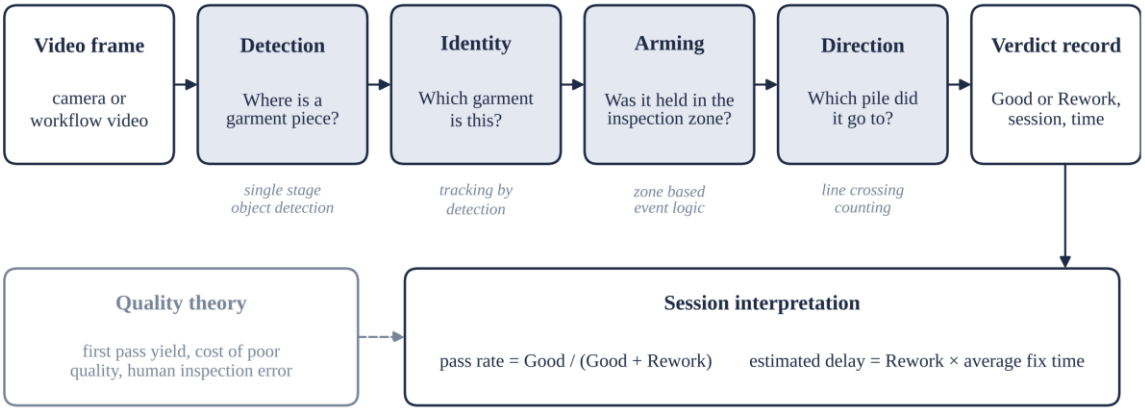


Figure 2.1: Conceptual framework from video frame to quality information

The framework makes the central research idea explicit. Each stage can fail in its own way, and only the last two stages are specific to this problem. The detector and tracker are standard and have well understood metrics; the arming and direction rules are new, have no benchmark, and are therefore the focus of the evaluation in Chapter 6.

3 RESEARCH METHODOLOGY

3.1 Research design

The research followed a design science approach [31]: build an artifact for a practical problem, evaluate it, and feed the lessons back into the design. It moved through four overlapping phases: understanding how decisions are made and recorded at a quality checking table; preparing the garment piece dataset and training the detector; developing the service, the verdict logic and the interface, first as a standalone desktop application and then as a feature of the integrated platform; and evaluating the result.

Evaluation used four kinds of evidence, each suited to a different claim. Claims about the detector rest on its validation metrics. Claims about the software, such as whether a verdict is stored once and only for the current session, rest on automated tests. Claims about repeatability rest on replays of a recorded workflow video. Claims about the robustness of the verdict logic rest on a controlled simulation in which synthetic garment trajectories with known verdicts are fed through the real counting code. Keeping these apart prevents the strength of one kind of evidence from being borrowed for another; in particular, high detector scores say nothing on their own about whether the counts are right.

3.2 Data participants or study materials

3.2.1 Garment piece detection dataset

The detector was trained on a dataset exported from Roboflow [32] under the name Garment Piece Detection, version 3 (qc_dataset-v3), in YOLOv8 format. It contains one class, Garment-Piece-Detection, annotated with boxes around whole garment pieces as they appear on and above the checking table. A single class was chosen on purpose: the component needs to know where a garment is, not what kind of garment it is or whether it is defective, and one class keeps annotation effort and confusion between classes to a minimum. Table 3.1 summarises the dataset.

Table 3.1: Composition of the garment piece detection dataset

Property	Value
Source and version	Roboflow export, Garment Piece Detection v3 (qc_dataset-v3), YOLOv8 format
Classes	1 (Garment-Piece-Detection)

Property	Value
Training images / instances	5385
Validation images / instances	385
Test images (if any)	384
Capture conditions	Overhead factory floor camera, top-down viewpoint, standard factory lighting conditions capturing garment pieces

3.2.2 Workflow video and stored sessions

The component can take its input from a webcam or from a workflow video file, which makes it possible to replay the same recorded inspection sequence several times. The seed database supplied with the component contains three sessions recorded on 28 August 2026 from such replays. These sessions, with their tracker identifiers, verdicts and times, are analysed in Section 6.2. No independent annotation of the video was kept, so the sessions support claims about repeatability and timing rather than accuracy.

3.2.3 Synthetic trajectories

To study the verdict logic under controlled conditions, a generator produced synthetic garment trajectories whose correct verdicts are known. Each garment appears in the inspection zone at a random position, stays there for 30 to 90 frames (one to three seconds at 30 frames per second), moves to the good or rework side over 5 to 12 frames, stays there for 2 to 8 frames and disappears. Eighty percent of garments are good. Four kinds of imperfection can be added independently: Gaussian jitter of the box centre, random loss of detections, identity switches in which the tracker gives the same garment a new identifier, and spurious short tracks that appear for one to six frames at random positions, standing in for hands, tools or reflections. Section 3.3 explains how these trajectories were driven through the real code.

3.2.4 Hardware

The detector was trained on a workstation with a CUDA graphics processor. The component itself runs on an ordinary Windows or macOS computer with a USB webcam; on Windows the camera is opened through DirectShow, falling back to the default OpenCV backend if that fails. The automated tests and the simulation were run on a Linux machine with no camera or graphics processor, which is possible because both replace only the camera and model boundary, as described next.

3.3 Experimental or study procedure

Detector training. YOLOv8n was fine tuned from COCO pretrained weights [21] with the Ultralytics toolkit [3], using the settings in Table 3.2. These values were read from the training arguments stored inside the trained checkpoint, so they are exactly those used. Training ran deterministically with seed 0, used mixed precision, and kept the default augmentation: mosaic until the last ten epochs, horizontal flips, colour jitter in HSV, translation, scaling, RandAugment and random erasing.

Table 3.2: Detector training configuration

Setting	Value
Architecture and initial weights	YOLOv8n (depth 0.33, width 0.25), pretrained on COCO
Toolkit version	Ultralytics 8.4.28
Epochs / batch size / image size	50 / 8 / 640 pixels
Optimiser and learning rate	Automatic selection; lr0 0.01, final factor 0.01, momentum 0.937, weight decay 0.0005, 3 warm up epochs
Loss weights	Box 7.5, classification 0.5, distribution focal 1.5
Augmentation	Mosaic 1.0 (closed for last 10 epochs), flip 0.5, HSV 0.015 / 0.7 / 0.4, translate 0.1, scale 0.5, RandAugment, erasing 0.4
Hardware and duration	One GPU, AMP enabled; 3,071 s in total (about 51 minutes)
Output checkpoint	models/garment-rework/best.pt (6,250,410 bytes), saved 15 August 2026

Iterative development. The service and interface were developed iteratively. After each change the system was run with the webcam and with workflow videos, and each unexpected behaviour was traced to its cause. Several important changes came from this process during integration: the video proxy in the desktop shell originally aborted the stream after ten seconds, which also stopped counting; the live view was originally unmounted when the user opened the Analysis tab, which closed the stream for the same reason; and the preview needed a reconnecting URL so that a restarted session would show a fresh stream. The first and third are now guarded by frontend tests, and the second is handled by keeping the live view mounted and only hiding it (Section 5.3).

Automated testing. A test harness imports the real service module and replaces only four external libraries with small stubs: the OpenCV camera interface, the Ultralytics model, the Supervision tracker and annotators, and the Uvicorn server. The stub model returns scripted detections frame by frame, each with a tracker identifier and a box centre, and the stub tracker passes them through unchanged. Everything else, including

the HTTP handlers, the zone logic, the state machine and the SQLite code, runs exactly as shipped. Each test uses a fresh temporary data directory supplied through the same environment variable the desktop shell uses.

Simulation study. The same harness was used to drive synthetic trajectories through the real frame loop. For each condition, three independent episodes of 200 garments each (600 garments in total) were generated with fixed random seeds, a session was started through the HTTP interface with the default settings, the frame generator was drained, and the stored verdicts were compared with the known verdicts. Table 3.3 lists the conditions. A second experiment evaluated a refinement that the current code does not have: requiring an armed track to remain in the exit zone for k consecutive observations before its verdict is logged. Because this needed a change to the logic, it was run on a separate Python model of the state machine; with k equal to one, this model produced exactly the same verdicts as the real code on five mixed noise episodes, which establishes that it reproduces the shipped behaviour.

Table 3.3: Conditions of the simulation study

Experiment	Varied factor	Levels	Other settings
S1 Baseline	None	Clean trajectories	Lines at 30 % and 70 %, centre band 10 % to 90 %
S2 Jitter	Standard deviation of the box centre	0, 2, 4, 6, 8, 10, 12 % of frame height	As S1
S3 Line gap	Line positions	40/60, 35/65, 30/70, 25/75, 20/80 %	Jitter 4 % and 8 %
S4 Dropout	Probability a detection is missing	0, 20, 40, 60, 80, 90 %	Jitter 2 %
S5 ID switch	Probability per frame of a new identifier	0, 1, 2, 5, 10 %	Jitter 2 %
S6 Spurious tracks	New spurious track per frame	0, 1, 2, 5, 10 %	Jitter 2 %
S7 Confirmation	Frames k required in the exit zone	1, 2, 3, 5, 8	Jitter 6 % and 8 %, dropout 60 %, switches 5 %, spurious 5 %

3.4 Measures and evaluation metrics

Detector quality is reported with precision, recall and average precision. A prediction is a true positive when its Intersection over Union with a ground truth box exceeds the threshold.

$$P = TP / (TP + FP), \quad R = TP / (TP + FN)$$
$$IoU = \text{area}(B_pred \cap B_gt) / \text{area}(B_pred \cup B_gt) \tag{3.1}$$

AP is the area under the precision and recall curve; with one class, mAP equals AP. It is reported at an IoU of 0.5 and averaged over IoU thresholds from 0.50 to 0.95 [21], [23]. For the counter, the box centre matters far more than the box edges, so mAP@0.5 is the more relevant of the two.

The zone of a tracked box with centre (c_x, c_y) in a frame of width W and height H follows from the line settings, given as percentages t, b, l and r for the top line, bottom line and the left and right edges of the centre band.

$$\text{zone} = \text{UP if } c_y < \lfloor H \cdot t/100 \rfloor;\ \text{DOWN if } c_y > \lfloor H \cdot b/100 \rfloor;\ \text{else MID}$$
$$\text{zone} = \text{MID whenever } c_x \notin [\lfloor W \cdot l/100 \rfloor, \lfloor W \cdot r/100 \rfloor] \tag{3.2}$$

A session's results are summarised by its counts G and K of Good and Rework verdicts, the pass rate, which corresponds to first pass yield [6], and the estimated delay, where f is the average fix time per rework garment entered by the user.

$$\text{pass rate} = 100 \times G / (G + K)$$
$$\text{estimated delay} = K \times f \ \text{(minutes)} \tag{3.3}$$

In the simulation, each garment has exactly one correct verdict. A garment is correct if the first record linked to any of its track identifiers has the correct verdict, wrong if it has the other verdict, and missed if it has no record. Additional records for the same garment are double counts, and records created by spurious tracks are extra records. The correct verdict rate is the share of garments counted correctly, and the counting error combines everything that changes the totals.

$$\text{correct rate} = \text{correct} / N$$
$$\text{counting error} = (\text{missed} + \text{double} + \text{extra}) / N \tag{3.4}$$

Software quality is measured by the test pass rate and statement coverage of the service module.

3.5 Data analysis

Detector results were analysed from the per epoch validation history stored in the checkpoint, looking at convergence, the gap between the two mAP figures and the balance of precision and recall. The stored sessions were analysed for their sequence of verdicts, tracker identifiers and timing. Simulation results were aggregated over 600 garments per condition and examined for the type of error, not only its rate, because a wrong verdict, a missed garment and an extra record have different consequences for a supervisor. Test results were analysed for which lines of the service remained unexecuted, since an unexecuted branch can reveal dead code as well as missing tests.

3.6 Ethics privacy security and responsible AI

Participants and consent. No personal data were collected for analysis. The automated tests and the simulation use synthetic data only. The workflow video replays were run by the author.

Privacy and fair measurement. The camera looks down at the checking table, but an inspector's hands and arms are inevitably in view. The live feed is not recorded, the database stores only a session name, a tracker number, a verdict and a time, and all processing stays on the workstation. The component counts decisions per session, never per person, and it must not be used to rank individual inspectors: a high rework count usually reflects the sewing that came before, not the person who found the defects. Because the inspector's decision is treated as the truth, the component also preserves the inspector's professional authority rather than overriding it with a model.

Security and responsible AI. The service listens only on the loopback interface, on a port chosen by the desktop shell at start up, and the renderer can reach it only through an allow list of five request paths. All SQL uses parameter binding, which a test confirms. The interface shows the annotated feed with box identifiers and lines, so an operator can see what the counter sees, and it offers a sensitivity setting and adjustable lines so that the behaviour can be corrected without code changes. The limitations found in this thesis are reported openly in Chapters 5 to 7.

3.7 Reproducibility and research artefacts

The repository holds the frontend, the backend, the trained checkpoint and the shared environment definition, with the OpenCV version pinned for reproducible camera behaviour. The training configuration and per epoch results are embedded in the checkpoint. The test suite and simulation scripts run on any machine with Python, FastAPI and NumPy, without a camera, a graphics processor or the model weights, and use fixed random seeds. The main limits to reproducibility are the unpublished dataset and workflow video.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements came from how decisions are made at a quality checking table, from what a line supervisor needs to see, and from the rules of the integrated platform. Table 4.1 lists the functional requirements and Table 4.2 the nonfunctional ones.

Table 4.1: Functional requirements of the component

ID	Requirement
FR1	Detect garment pieces on and above the checking table in each frame from a webcam or a workflow video.
FR2	Keep a stable identity for each garment while it is in view.
FR3	Record a Good verdict when an inspected garment is moved to the good side and a Rework verdict when it is moved to the rework side.
FR4	Record each garment at most once, and never record a garment that was not inspected in the middle zone.
FR5	Store each verdict with its session, tracker identifier and time.
FR6	Start and end sessions, with counts that start from zero in each new session.
FR7	Let the operator choose a camera or a workflow video, and adjust sensitivity, line positions and the centre band during a session.
FR8	Show the annotated live feed, Good and Rework counts, total checked, pass rate and estimated order delay.
FR9	Show trend and verdict split charts, the current session log and a history of all sessions with pass rate and delay.
FR10	Export the session log and the session history as CSV files.

Table 4.2: Nonfunctional requirements of the component

ID	Requirement
NFR1	Performance: process frames at an interactive rate on the workstation and show a verdict on screen within about one second.
NFR2	Zero added work: the inspector must not press, scan or say anything to record a decision.
NFR3	Isolation: run as an independent service with its own model and database, never calling another feature's backend.
NFR4	Security: listen only on the local machine and accept only allowed requests from the interface.
NFR5	Usability: start from a workstation card with clear progress and error messages, and release the camera when leaving the feature.
NFR6	Transparency: show what the counter sees, including boxes, identifiers and lines.
NFR7	Portability: keep all data in one local file outside the installed application so updates cannot overwrite results.
NFR8	Maintainability and testability: keep the logic testable without camera or model hardware.

4.2 Component architecture within the complete solution

The component runs as two cooperating processes inside the desktop platform, shown in Figure 4.1. The rework service (FastAPI [33], OpenCV [34], Ultralytics and Supervision) owns the camera or video, runs detection and tracking, applies the verdict logic, stores verdicts in SQLite and serves both its REST endpoints and an MJPEG stream. The Electron main process starts that service and mediates every interaction with it. The React renderer presents the two views and talks only to the main process.

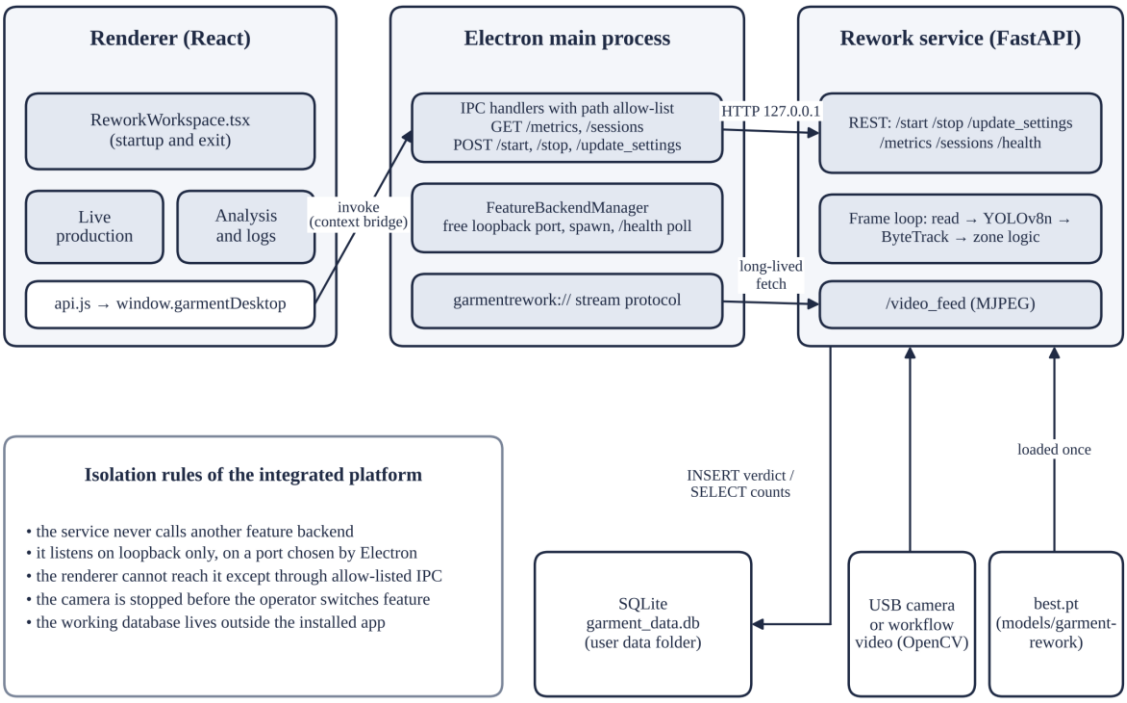


Figure 4.1: Process architecture of the Rework Counter inside the platform

The service is started by a generic feature backend manager that the platform also uses for the size feature. On start it asks the operating system for a free port on 127.0.0.1, sets environment variables that tell the service where its model and data directory are, and launches the service: in development through the shared Python environment with Uvicorn, and in an installed build as a bundled executable. It then polls the health endpoint every 350 milliseconds, with a deadline of 60 seconds in development and 120 seconds in an installed build, because loading the model can be slow on first use. While the service starts, the interface shows a shared startup screen for the Quality Checking area with a retry button.

This arrangement satisfies the isolation rules of the platform (NFR3, NFR4). The service never knows about the other features; the renderer never knows the service's port; and the main process forwards only requests whose method and path are on an allow list. The working database lives in the user data folder, under features/rework, rather than inside the installed application (NFR7). When the operator leaves the feature, the workspace first asks the main process to stop the camera, so that the next feature can open it.

4.3 Design alternatives and decisions

Table 4.3 summarises the main design choices. The most important are explained below.

Table 4.3: Main design decisions and the alternatives considered

Concern	Alternatives	Decision	Main reason
Source of the verdict	Defect classifier; button or tablet entry; movement direction	Movement direction	No defect dataset needed; no extra action; the inspector stays the judge
Detector	Multi class garment detector; larger YOLO; two stage detector	YOLOv8n, one class	Real time on modest hardware; only position is needed
Tracker	Centroid matching; SORT; DeepSORT; ByteTrack	ByteTrack via Supervision	Keeps tracks through partial occlusion by hands; no appearance model
Counting rule	Single line crossing; two lines; two lines with inspection zone	Two lines with an arming zone	Ignores garments that were never inspected and garments already on the piles
Anchor point	Box edge; bottom centre; box centre	Box centre	Symmetric for both directions
Storage	In memory only; CSV files; SQLite	SQLite, one row per verdict	Durable, queryable by session, single file
Live updates	WebSockets; server sent events; polling	Polling every second plus MJPEG	Simple, adequate for one workstation
Integration	Merge into a shared backend; separate service per feature	Separate service per feature	Independent ownership, failure isolation and testing

Source of the verdict. The decisive choice was not to classify defects. A defect classifier would need many labelled images of each kind of defect on each kind of fabric, would be hard to validate, and would put the model's judgement above the inspector's. Manual entry would be accurate but would add a step to every garment, which is exactly the burden the component is meant to remove (NFR2). Reading the decision from the direction in which the inspector moves the garment needs only a

garment detector, adds no work, and keeps the human decision as the source of truth. Its price is that the counter can only be as good as its interpretation of movement, which is why so much of the evaluation concerns that interpretation.

Detector and tracker. Because the verdict comes from movement, the detector only has to answer where a garment is. A single class YOLOv8 nano model [3] is enough for this and is small enough to run alongside the rest of the platform. ByteTrack [4] was chosen over simpler trackers because hands repeatedly cover parts of the garment during inspection, which lowers detection confidence; ByteTrack's use of low confidence detections keeps the track alive through these moments. It is configured with an activation threshold of 0.20 and keeps a lost track for up to 120 frames, about four seconds at 30 frames per second, so that a garment briefly hidden by the inspector's body keeps its identity.

Counting rule. A single line would count any garment that crosses it, including garments carried past the table. Two lines with a central inspection zone add a necessary condition: a garment can only produce a verdict if it was first seen in the middle zone, inside the centre band. This one rule removes several classes of false count. Garments lying on the piles are already in the UP or DOWN zone when they are first seen and are never armed; new identities given to a garment on a pile after it has been counted are equally ignored; and objects that only pass along the edges of the view never enter the centre band.

4.4 Detailed design

4.4.1 Frame loop

All counting happens in one frame loop inside the service. For each frame the loop reads the next image from the capture, runs the detector at the configured confidence threshold, converts the result to Supervision detections and passes them to ByteTrack, which returns the same boxes with tracker identifiers. For every tracked box the loop computes the centre, determines its zone with the current line settings (Equation 3.2), and updates the state of that track. It then draws the boxes, the identifiers, the two horizontal lines and the two vertical centre band lines on a copy of the frame, appends

the current counts to an in memory history, encodes the frame as JPEG and yields it as one part of the MJPEG stream.

The loop is a Python generator that is consumed by the video feed endpoint. This has an important consequence: frames are read and counted only while a client is reading the stream. In the platform that client is the image element of the Live production view, through the desktop shell's stream protocol. The design keeps the service very simple, with no separate thread and no locking, but it couples counting to the preview, which is examined in Sections 5.3 and 7.5.

4.4.2 Zone geometry

Figure 4.2 shows the geometry of the zones with the default settings and Table 4.4 lists the parameters. The top line is at 30 % of the frame height and the bottom line at 70 %, so the inspection zone occupies the middle 40 % of the frame. The centre band spans 10 % to 90 % of the width; a box centre outside the band is always treated as MID, whatever its height, so movements along the sides of the view can neither arm a track nor produce a verdict. All five values can be changed during a session with sliders whose ranges keep the lines in their own halves of the frame.

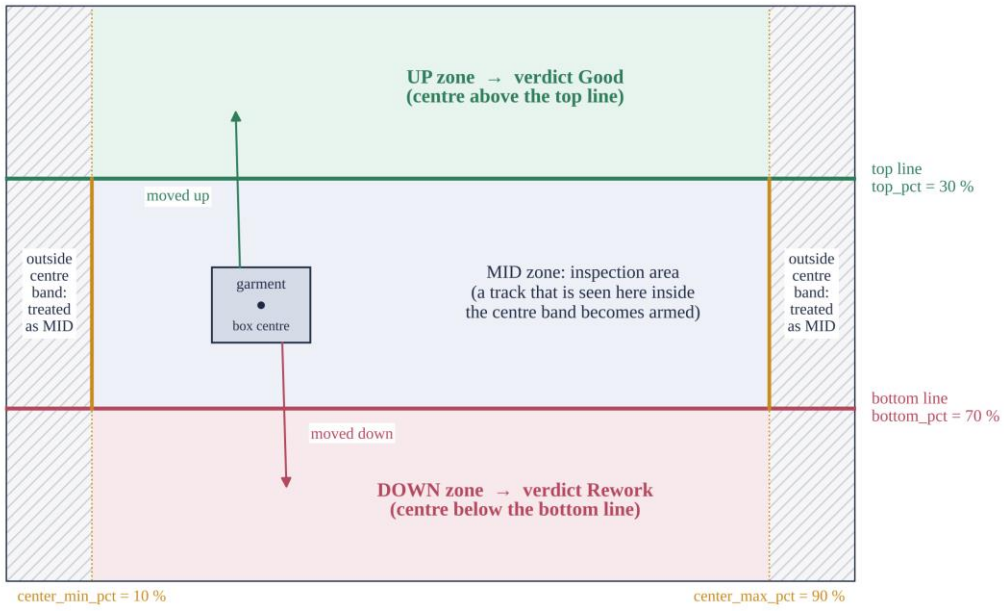


Figure 4.2: Zones of the frame with the default line settings

Table 4.4: Parameters of the detection, tracking and verdict logic

Parameter	Default	Range in the interface	Function
conf_threshold	0.25	0.05 to 0.95	Minimum detector confidence (labelled Detection sensitivity)
top_pct	30 %	5 % to 50 %	Position of the top (Good) line
bottom_pct	70 %	50 % to 95 %	Position of the bottom (Rework) line
center_min_pct	10 %	0 % to 40 %	Left edge of the centre band
center_max_pct	90 %	60 % to 100 %	Right edge of the centre band
track_activation_threshold	0.20	Fixed	ByteTrack confidence to start a track
lost_track_buffer	120 frames	Fixed	How long a lost track can be recovered
Average fix time	2.0 min	Steps of 0.5 min	Minutes of rework per Rework verdict, for the delay estimate

4.4.3 Per track state machine

Each tracker identifier has a small state record holding its last zone, whether it has visited the middle zone, whether it has been counted, and a counter of missed frames. Figure 4.3 shows the resulting state machine. A new identifier starts unarmed. It becomes armed the first time its centre is in the MID zone and inside the centre band. An armed, uncounted track that is seen in the UP zone is counted as Good, and one seen in the DOWN zone as Rework; the verdict is written to the database at once and the track is marked counted, after which nothing it does can change or repeat its verdict.

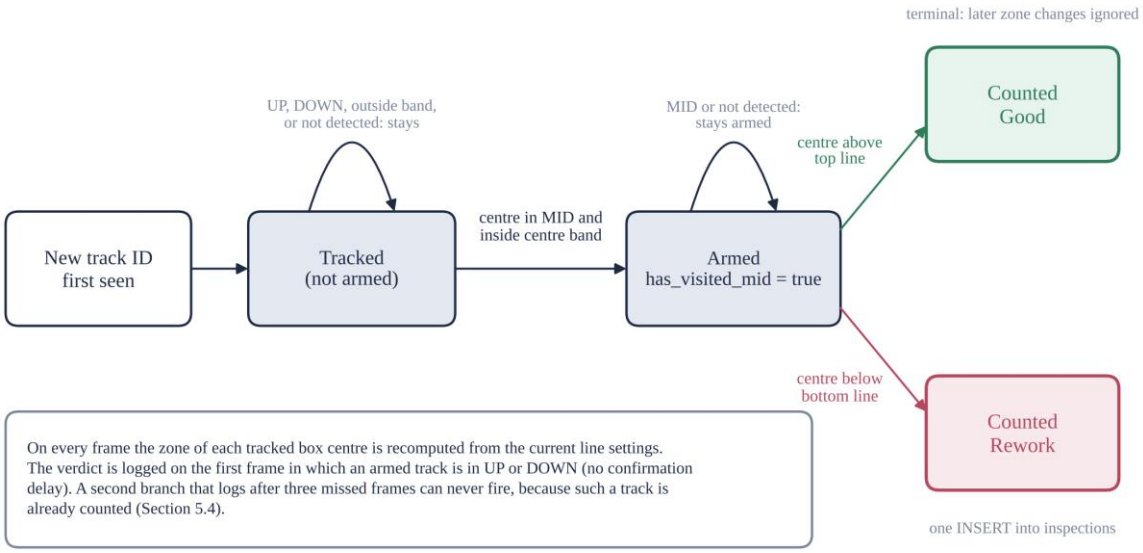


Figure 4.3: State machine applied to every tracker identifier

The code also contains a second path to a verdict: an armed, uncounted track that disappears for three consecutive frames while its last zone was UP or DOWN is counted on that basis. The intention was to catch a garment that is placed on a pile and immediately covered. Analysis of the state machine shows that this path can never be taken, because a track whose last zone is UP or DOWN was already counted in the frame in which it entered that zone. The coverage results in Section 5.4 confirm that these lines are never executed. They do no harm, but they create the impression of a safeguard that does not exist.

4.4.4 Session model and persistence

A session begins when the operator presses Start New Session and ends with End Session, with the end of a workflow video, or when the operator leaves the feature. Starting a session opens the selected source, creates a new tracker, clears all track states, counts and history, and names the session from the local date and time, for example SESSION_20260828_061611. The model is loaded the first time a session starts and then kept for the life of the service, so later sessions start quickly.

The data model is a single table, shown in Figure 4.5. Each verdict is one row holding the session name, the tracker identifier, the verdict and the local clock time. Live counts are computed from the table for the current session only, so that a restarted interface cannot show a stale total, while the per frame history used for the trend chart is kept in memory, capped at 500 points, and served as its last 100 points. The session history groups the table by session, computes the pass rate in the service and leaves the delay to the interface, which multiplies the Rework count by the average fix time the user has entered.

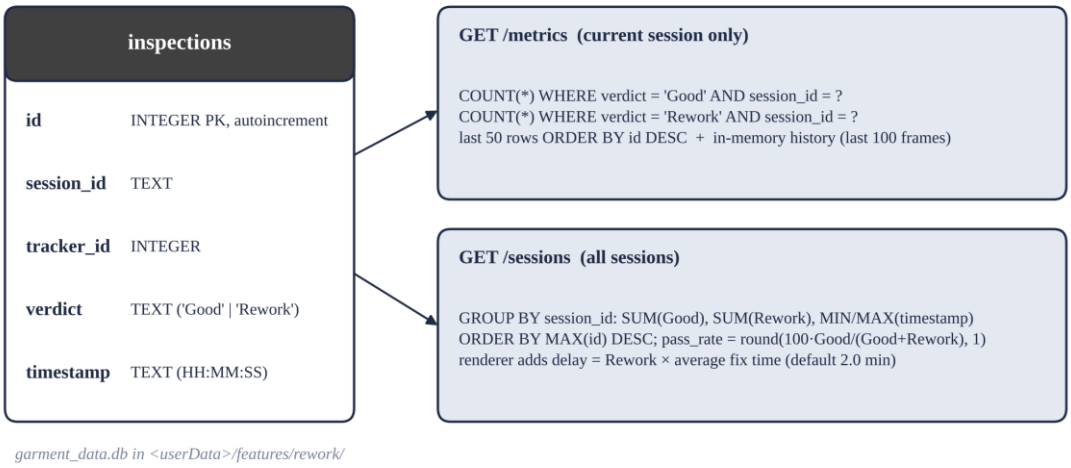


Figure 4.4: Data model and the two queries that serve the interface

Table 4.5 lists the endpoints. Request bodies for starting a session and updating settings are validated by Pydantic models before the handler runs, so a malformed request is rejected with HTTP 422.

Table 4.5: Service endpoints

Method	Path	Purpose	Allowed from the renderer
GET	/health	Readiness probe with model and database paths	No (main process only)
POST	/start	Open the source and start a new session	Yes
POST	/update_settings	Change sensitivity and lines during a session	Yes
POST	/stop	End the session and release the source	Yes
GET	/metrics	Counts, last 50 verdicts, history, running state	Yes
GET	/sessions	Summary of every session	Yes
GET	/video_feed	Annotated MJPEG stream; drives the frame loop	Through the garmentrework protocol

4.4.5 Session lifecycle

Figure 4.4 shows the interactions over a complete session. Once the service is ready, starting a session is a single allowed request. The Live production view then sets its image source to a fixed address on the garmentrework protocol, which the main process turns into a long lived request to the video feed. Every second the view asks for the metrics; the Analysis view asks for metrics every two seconds and for the session history every five seconds. Changes to the sliders are sent to the service 200

milliseconds after the last movement, so that dragging a slider does not flood the service with requests.

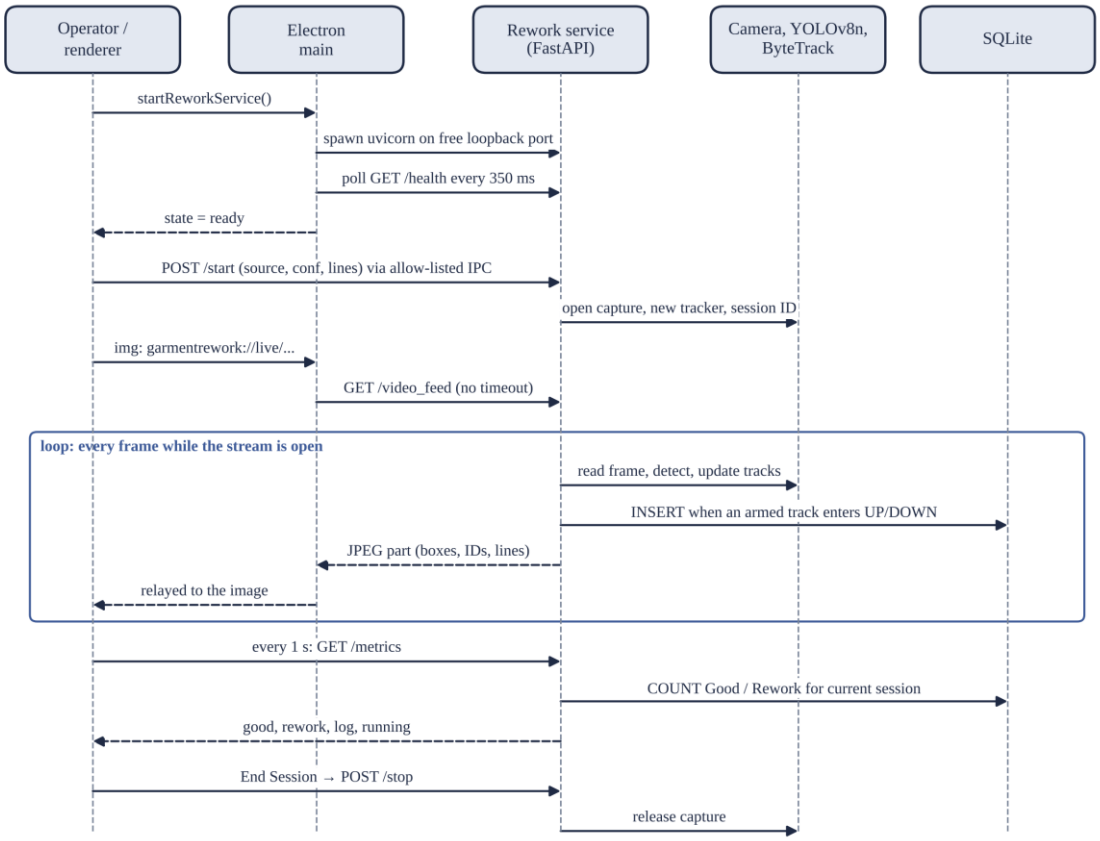


Figure 4.5: Interactions during one Rework Counter session

4.4.6 Interface design

The Live production view has three columns. On the left are the input controls, with a choice between webcam and video file, a camera list built from the browser's device labels with a manual index fallback, a button to choose a workflow video, the start and end buttons, the average fix time and the advanced settings. In the centre is the annotated feed, with a status badge that distinguishes a live feed, a preview that is still connecting and a disconnected camera. On the right are five large figures: Good, Rework, estimated order delay, total checked and pass rate. A badge at the top shows whether inspection is running and the name of the active session.

The Analysis and logs view shows a line chart of cumulative Good and Rework counts over frames, a bar chart of the current split, the current session log with the newest verdict first, and the history of all sessions with their counts, pass rate, delay and time

span. Pass rates above 90 % are shown in green, above 75 % in amber and otherwise in red. Both tables can be exported as CSV. The two views are kept mounted and only hidden when the user switches tab, so the preview and therefore the counting continue while the user reads the analysis.

4.5 Standards and professional practice

The service follows conventional HTTP semantics, with validation errors returned as 422 and the health endpoint used as a readiness probe, and FastAPI generates OpenAPI documentation for it automatically [33]. All database statements use parameter binding. The desktop shell keeps Electron's context isolation and exposes only a small preload interface [35], and the renderer's content security policy names the garmentrework scheme explicitly as an allowed image source. The release pipeline records a SHA-256 checksum of the model file and bundles the service with its dependencies, so that an installed build does not depend on the user's Python installation. Work was tracked in Microsoft Teams and Planner with a weekly log book, and integration rules were written down in the platform guide shared by the group.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The component was developed on a Windows laptop, the same class of machine it targets, and is built into the platform installer for Windows and macOS. Table 5.1 lists the technologies used.

Table 5.1: Technology stack of the component

Layer	Technology	Reason for selection
Desktop shell	Electron 43 with Electron Forge and webpack	Native window, process supervision, custom protocols
User interface	React 19, Tailwind CSS 4, Recharts 3, Lucide icons	Component model, consistent styling, simple charts
Service	FastAPI with Uvicorn and Pydantic (Python 3.12)	Validated endpoints, streaming responses, small footprint
Vision	OpenCV 4.11.0.86 (pinned), Ultralytics 8.x	Camera and video input, YOLOv8 inference
Tracking and drawing	Supervision (ByteTrack, box and label annotators)	Maintained ByteTrack implementation with a simple interface
Persistence	SQLite (Python standard library)	Zero configuration, one file, transactional
Packaging	PyInstaller sidecar per feature; platform installer	Operators need neither Python nor Node.js
Testing	pytest with FastAPI TestClient and coverage; Node test runner	Service, logic and persistence tests; frontend source tests
Project management	Microsoft Teams and Planner	Task tracking and weekly log book

In the platform repository the component occupies three folders: frontend/garment-rework-frontend for the views and the desktop adapter, backend/garment-rework-backend for the service and its seed database, and models/garment-rework for the trained checkpoint. The folders belong to the component's owner; the shared Electron code under frontend/src only starts the service and relays requests.

5.2 Component implementation

Detector. The model was trained with the Ultralytics interface and saved as best.pt (6.25 MB). The release pipeline copies it into the installed application, and the desktop shell passes its path to the service in the GARMENT_REWORK_MODEL_PATH environment variable, so the service never assumes where it is installed. The model is loaded once, on the first session start.

Input sources. A session can use a webcam, identified by an OpenCV index, or a workflow video file. On Windows a webcam is opened through DirectShow and, if

that fails, through OpenCV's default backend, and the capture buffer is set to one frame so that the counter always works on the newest image. A video file is checked for existence before it is opened, and when it reaches its end the loop stops, releases the capture and marks the session as no longer running; the interface notices this at its next metrics request and shows that the workflow video has completed.

Verdict logic. The heart of the service is the per track update inside the frame loop, shown below in condensed form; new_state() stands for the initial record with the zone set to MID and both flags false. It is a direct implementation of the state machine in Figure 4.3.

```
for tid, (x1, y1, x2, y2) in tracked_boxes:
    cx, cy = (x1 + x2) / 2, (y1 + y2) / 2
    zone = 'MID'
    if cy < top_y: zone = 'UP'
    elif cy > bottom_y: zone = 'DOWN'
    in_band = center_x_min <= cx <= center_x_max
    if not in_band: zone = 'MID'
    s = track_states.setdefault(tid, new_state())
    if zone == 'MID' and in_band:
        s['has_visited_mid'] = True
    s['zone'] = zone
    if (not s['counted'] and s['has_visited_mid']
            and zone in ('UP', 'DOWN')):
        s['counted'] = True
        verdict = 'Good' if zone == 'UP' else 'Rework'
        log_verdict(tid, verdict)
```

The function log_verdict increments the in memory counters and inserts one row into the inspections table with the current session name, the tracker identifier, the verdict and the clock time. Each insert opens and closes its own SQLite connection, which is simple and safe at the rate of a few verdicts per minute.

Live settings. The five detection and line settings are held in the service state and read again on every frame, so a change made with a slider takes effect on the next frame. The interface waits 200 milliseconds after the last slider movement before sending the update.

Views. The Live production view polls the metrics endpoint every second while a session runs and updates the five figures. It computes the pass rate and the estimated delay itself from the counts and the fix time. If the preview image fails, it schedules a reconnection after 900 milliseconds with a new URL containing the session name and a time stamp, which forces Chromium to open a fresh stream. The Analysis view draws

the charts from the history returned by the metrics endpoint and builds the CSV files in the browser from the session log and the session history.

5.3 Integration with the other components

The Rework Counter is integrated through the platform's generic feature backend manager, its IPC layer and a dedicated stream protocol, and shares nothing else with the other three components. The interfaces are small and explicit.

- **Start and status.** The workspace calls startReworkService through the preload bridge; the main process allocates a port, launches the service with its model path, data directory and Ultralytics configuration directory, and polls /health until it answers.
- **Requests.** The renderer calls reworkRequest with a method and a path. The main process checks the pair against an allow list (GET /metrics and /sessions; POST /start, /stop and /update_settings) and rejects anything else before it reaches the service.
- **Video.** The renderer receives the fixed address garmentrework://live/video.mjpeg. The main process's protocol handler accepts only that address and opens a long lived request to /video_feed, whose body it passes straight through.
- **Workflow video selection.** A dedicated dialog restricted to common video formats (mp4, mov, avi, mkv, m4v and webm) returns the chosen path, which the renderer sends with the start request.
- **Leaving the feature.** The workspace calls stopReworkCamera before returning to the landing screen, which sends /stop if the service is ready, so the next feature finds the camera free.

Three integration problems were found and fixed during this work, and each shows how closely the counting depends on the stream. First, the main process originally fetched the stream with the same ten second timeout it uses for ordinary requests, so the preview disappeared after ten seconds; because the frame loop runs only while the stream is read, counting stopped at the same moment. The stream is now fetched without a timeout. Second, the renderer's content security policy had to name the garmentrework scheme as an image source, without which the preview is blocked.

Third, switching between the Live production and Analysis tabs originally unmounted the live view, closing the stream; the views are now kept mounted and only hidden. The first two fixes and the reconnecting preview are guarded by frontend tests.

Integration was verified by the frontend tests in Section 5.4, by the platform's own layout tests, which check that every component has its own frontend, backend and model folder and that all four backends are installed into the shared environment, and by running the integrated application with both a webcam and a workflow video.

5.4 Verification and software testing

5.4.1 Test strategy and suite

Testing focused on the parts whose correctness can be decided exactly: the HTTP interface, the verdict logic and the persistence layer. Before this work the component had no backend tests. The new suite imports the real service module with only the camera, model, tracker and server libraries replaced by stubs, as described in Section 3.3, so every test exercises the shipped code. Scripted detections place a garment's box centre at chosen fractions of the frame in chosen frames, which makes every expected outcome unambiguous. Table 5.2 summarises the suite.

Table 5.2: Structure of the automated test suite

Suite	Tests	Focus
Interface tests (IT)	21	Health, input validation, session start and stop, live settings, stream, per session metrics, history ordering, log limit, model reuse, end of video
Counting logic tests (UT_C)	23	Good and Rework verdicts, arming rule, exactly once, first verdict final, centre band, line boundaries, live line changes, lost tracks, ID switches, jitter, history
Persistence tests (UT_D)	7	Schema, idempotent start up, data directory, timestamp format, parameter binding, midnight and same second sessions
Frontend tests (Node)	3	Stream allowed by content security policy, no stream timeout, workflow video picker and reconnecting preview
Total	54	

Several tests are deliberately written to document a known limitation rather than a desired behaviour; their names contain known_limit. They pass because they assert what the code does today, so that a future fix will make them fail and force the documentation to be updated.

5.4.2 Representative test cases

Table 5.3 lists selected cases; the complete list is in Appendix A.

Table 5.3: Representative test cases and their outcomes

ID	Scenario	Expected result	Outcome
UT_C01	Track seen in MID, then above the top line	One Good record	Pass
UT_C03	Track only ever seen on the good pile	No record	Pass
UT_C05	Track moves up, back to MID and up again three times	Exactly one Good record	Pass
UT_C06	Track moves down, then back up	One Rework record; first verdict is final	Pass
UT_C07	Exit above the top line but outside the centre band	No record	Pass
UT_C10	Centre exactly on the top line, and just above it	Only the second is counted	Pass
UT_C14	Frame by frame inspection of the loop	Verdict written in the first frame in the exit zone	Pass
UT_C16	Tracker gives a new ID while the garment is in MID	Counted once under the new ID	Pass
UT_C17	New ID after counting, garment taken back to MID and out again	Second record (known limit)	Pass
UT_C18	Centre jitters across the top line for one frame	Good recorded at once (known limit)	Pass
IT_15	Two sessions, one second apart	Metrics show only the current session	Pass
IT_21	Workflow video reaches its end	Session marked stopped, capture released	Pass
UT_D05	SQL injection text as a session name	Stored as plain data	Pass
UT_D07	Two sessions started within the same second	Same session name, counts merged (known limit)	Pass

5.4.3 Execution, coverage and defects

All 51 backend tests passed in about four seconds, and the three frontend rework tests passed with the Node test runner. Statement coverage of the service module, in Table 5.4, is 94 %: 230 of 244 statements were executed. The 14 statements not executed are the Windows DirectShow branch, which cannot run on the Linux test machine, the defensive release of a stale capture, the general exception handler of the start endpoint, the line that starts the server when the module is run directly, and the two lines of the missing frame fallback described in Section 4.4.3. The last two are unexecuted for a different reason from the others: no input can reach them.

Table 5.4: Statement coverage of the service module

Module	Statements	Missed	Coverage
backend/garment-rework-backend/main.py	244	14	94 %

Writing the tests and the simulation exposed the defects and limitations in Table 5.5. None of them affects the basic behaviour verified above, but several matter for a long running deployment and are taken up again in Chapter 7.

Table 5.5: Defects and limitations found during verification

ID	Finding	Evidence	Effect
F1	Missing frame fallback can never fire	Coverage; state machine analysis	No effect today; misleading safeguard
F2	Counting runs only while the stream is read	Frame loop is the stream generator	No preview, no counting; two viewers would share one capture
F3	Verdict logged on the first frame in the exit zone	UT_C14, UT_C18; simulation S2	Jitter near a line produces early or wrong verdicts
F4	New ID after counting can be counted again	UT_C17	Double count if the garment is handled again
F5	Session names have one second resolution	UT_D07	Two quick restarts merge into one session
F6	Times stored as clock time without date	UT_D04, UT_D06	Sessions across midnight show reversed start and end
F7	Track states are never removed during a session	Code review	Memory grows slowly in very long sessions
F8	Line settings are not validated in the service	Code review	Only the slider ranges keep the top line above the bottom line
F9	Fix time is entered separately in each view and not stored	Code review	Delay figures can differ between views and are lost on restart

5.5 Deployment and operation

In an installed build the operator needs no Python or Node.js: the release pipeline builds the service as a standalone executable that includes the Supervision package and its data, copies the model, and records its checksum. The first time the Rework Counter card is opened, the operating system asks for camera access. The operator then chooses the camera, checks that the annotated feed shows the two lines across the table with the good pile above the top line and the rework pile below the bottom line, adjusts the lines if needed, and presses Start New Session. From then on the inspector works as usual. The supervisor reads the counts, pass rate and delay on the Live production view and the history on the Analysis view, and can export either as CSV. Ending the session or leaving the feature releases the camera.

Camera placement matters more than any software setting. The camera should look straight down, the inspection position should be near the middle of the view, the piles

should be clearly above and below the lines, and nothing else should be moved through the middle of the view and out across a line. The results in Chapter 6 show why these conditions matter.

6 RESULTS AND VALIDATION

6.1 Research results

6.1.1 Detector training results

Figure 6.1 shows the validation metrics over the 50 training epochs. mAP@0.5 rose from 0.420 after the first epoch to 0.823 after ten, first passed 0.90 at epoch 28 (0.905), and finished at 0.938, with its highest value, 0.940, at epoch 49. The stricter mAP@0.5:0.95 climbed more slowly, from 0.173 to 0.556, and was still rising at the end of training. Because Ultralytics selects the best checkpoint by a fitness score dominated by mAP@0.5:0.95, the saved model is the one from the final epoch.

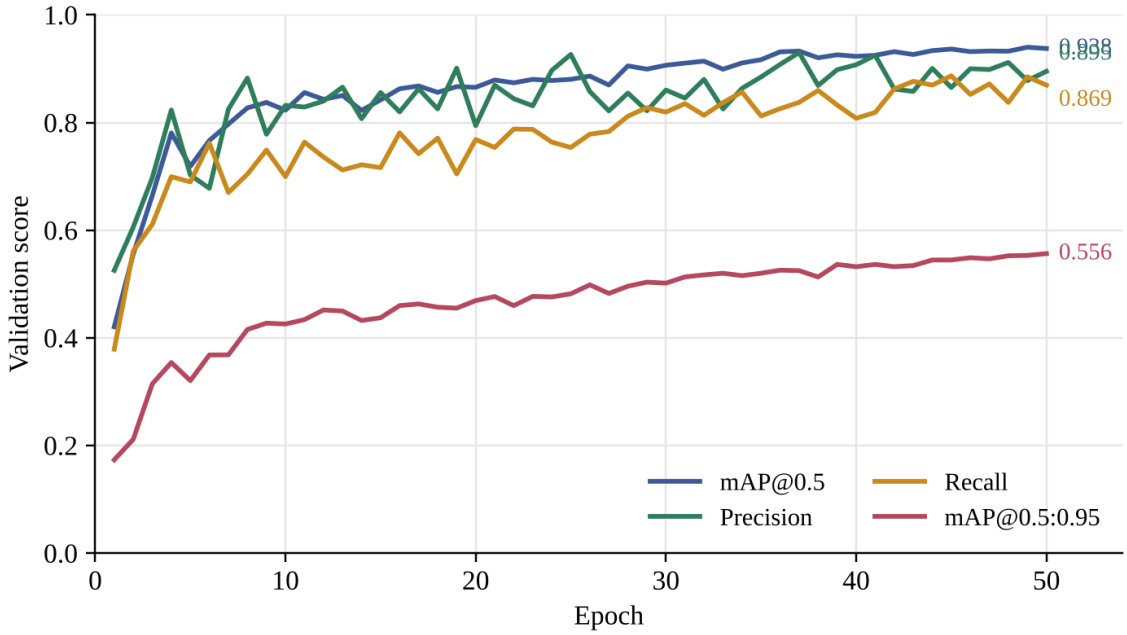


Figure 6.1: Validation metrics of the garment piece detector during training

Table 6.1: Validation results of the detector at selected epochs

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
1	0.525	0.379	0.420	0.173
10	0.832	0.700	0.823	0.426
25	0.927	0.754	0.880	0.482
40	0.907	0.808	0.923	0.532
50	0.895	0.869	0.938	0.556

At the final epoch the detector reached a precision of 0.895 and a recall of 0.869. In detection terms RQ1 is answered positively: a nano model fine tuned on one class locates garment pieces reliably. The large gap between mAP@0.5 (0.938) and

mAP@0.5:0.95 (0.556) shows that most remaining error lies in the tightness of the boxes rather than in finding the garments. A garment held and folded by hands has no crisp outline, so this is expected, and it matters little for the counter, which uses only the box centre. A box that is too large or too small by a few percent moves its centre much less than it changes its IoU.

Figure 6.2 shows the losses. All three training losses fell steadily. The validation box and distribution focal losses fell more slowly and flattened after about epoch 35, ending at 1.503 and 1.396, while the validation classification loss fell from 2.421 to 0.739. The widening gap between training and validation box loss in the last epochs is a mild sign that further training would mainly fit the training boxes more closely, not improve the counter.

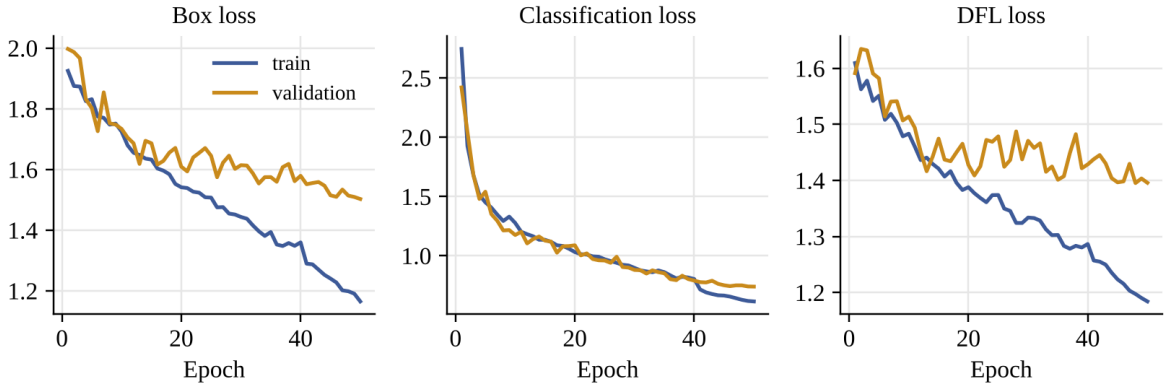


Figure 6.2: Training and validation losses of the detector

6.1.2 Verdict logic under controlled conditions

With clean trajectories (S1), the real counting code recorded all 600 garments exactly once with the correct verdict: no wrong verdicts, no misses, no double counts. This confirms on a larger scale what the unit tests show case by case, and it answers the first half of RQ2: when the tracker delivers a clean track, the zone rule records the inspector's decision correctly and exactly once.

Centre jitter. Figure 6.3 and Table 6.2 show the effect of jitter in the box centre while the garment is held in the inspection zone (S2). Up to a standard deviation of 2 % of the frame height the logic was still perfect, and at 4 % it was 99.0 % correct. Beyond that the correct rate fell quickly, to 85.5 % at 6 % and 64.0 % at 8 %, and approached the 50 % that a coin would achieve. All of these errors were wrong verdicts, never

misses: a garment whose centre briefly crossed a line while it was still being inspected was counted at once, on the side it happened to touch.

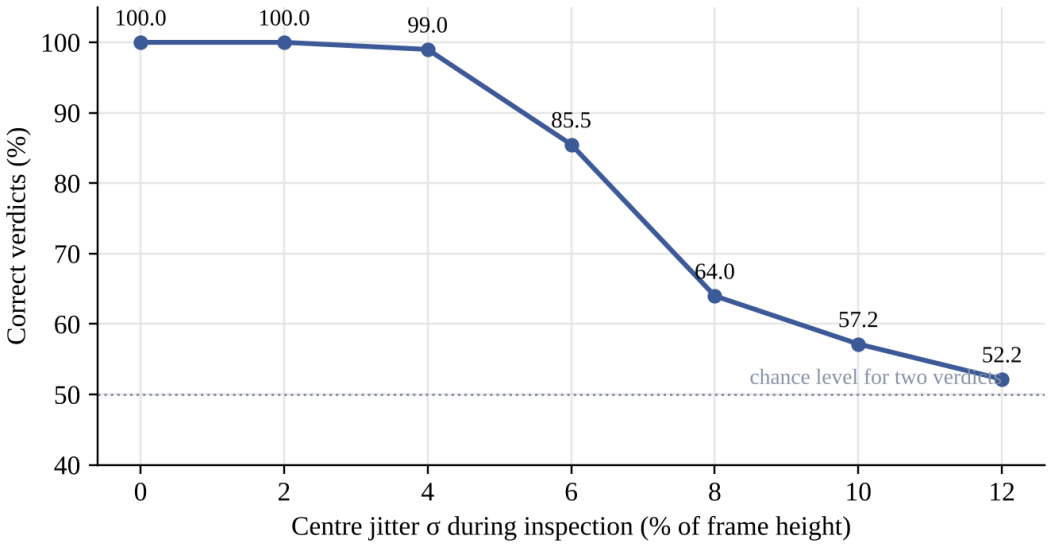


Figure 6.3: Correct verdicts as a function of centre jitter with the default lines

Table 6.2: Effect of centre jitter on the verdict logic (600 garments per level)

Jitter σ	Correct	Wrong	Missed	Correct rate
0 %	600	0	0	100.0 %
2 %	600	0	0	100.0 %
4 %	594	6	0	99.0 %
6 %	513	87	0	85.5 %
8 %	384	216	0	64.0 %
10 %	343	257	0	57.2 %
12 %	313	287	0	52.2 %

Line positions. The size of the inspection zone determines how much jitter the logic can absorb (S3). Figure 6.4 shows that moving the lines outwards from 40 %/60 % to 20 %/80 % raised the correct rate at 4 % jitter from 61.0 % to 99.8 %, and at 8 % jitter from 50.0 % to 97.2 %. The default lines at 30 % and 70 % are adequate for small jitter but leave little margin. The cost of wider lines is that the inspector must move the garment further before it counts, and with lines at 20 % and 80 %, one garment out of 600 at 4 % jitter was missed because it was placed just short of the bottom line.

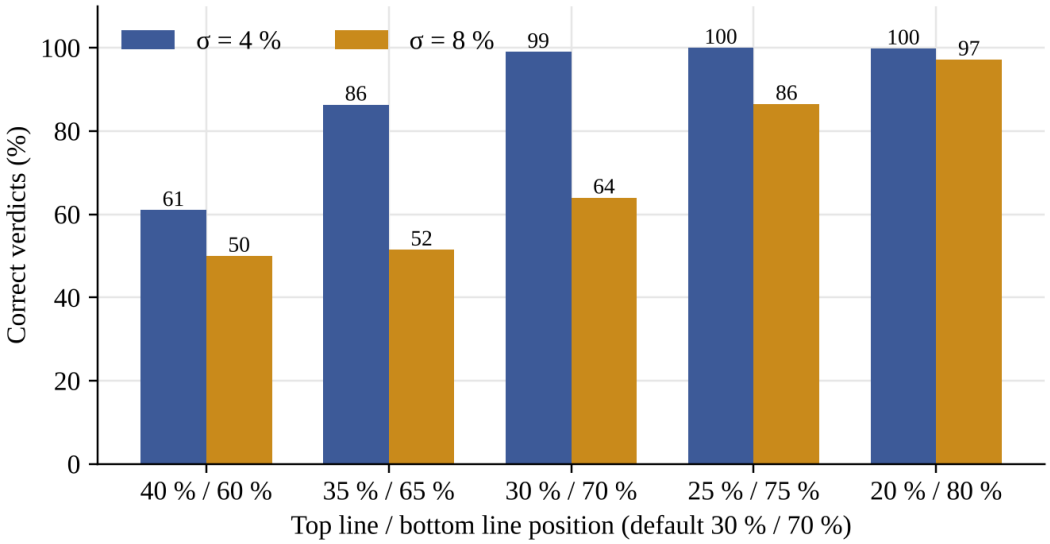


Figure 6.4: Correct verdicts for different line positions at two jitter levels

Missed detections, identity switches and spurious tracks. Figure 6.5 and Table 6.3 show the other three kinds of imperfection (S4 to S6). Missed detections were absorbed well: even when 40 % of detections were missing, only one garment in 600 was missed, because the tracker keeps the identity and the garment stays in the exit zone for several frames, any one of which is enough. Misses grew steeply only beyond 60 % dropout. Identity switches were more damaging than their rate suggests. With a switch probability of 1 % per frame, 0.8 % of garments were missed, and at 5 % per frame, 4.7 %. The mechanism is precise: when the switch happens during the exit movement, after the garment has left the middle zone, the new identity is first seen in the exit zone, is never armed, and so is never counted. Spurious tracks produced extra records at a rate that grew almost linearly with their frequency, from 5.7 extra records per 100 garments at one spurious track per 100 frames to 30 at five. They are counted when a short lived track happens to start in the middle zone and drift across a line.

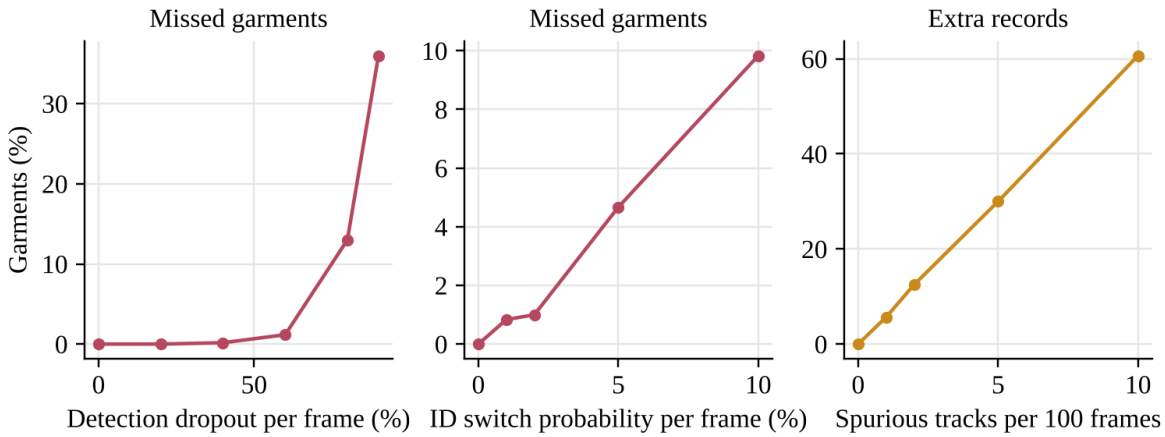


Figure 6.5: Missed garments under detection dropout and identity switches, and extra records from spurious tracks

Table 6.3: Effect of detection and tracking imperfections (600 garments per level, jitter 2 %)

Imperfection	Level	Correct	Missed	Double	Extra	Counting error
Dropout	20 %	600	0	0	0	0.0 %
Dropout	40 %	599	1	0	0	0.2 %
Dropout	60 %	593	7	0	0	1.2 %
Dropout	80 %	522	78	0	0	13.0 %
Dropout	90 %	384	216	0	0	36.0 %
ID switch	1 %	595	5	0	0	0.8 %
ID switch	2 %	594	6	0	0	1.0 %
ID switch	5 %	572	28	0	0	4.7 %
ID switch	10 %	541	59	0	0	9.8 %
Spurious	1 %	600	0	0	34	5.7 %
Spurious	2 %	600	0	0	75	12.5 %
Spurious	5 %	600	0	0	180	30.0 %
Spurious	10 %	600	0	0	364	60.7 %

Two results in Table 6.3 deserve attention because they show the arming rule working. Across all levels of identity switches there were no double counts, and across all levels of spurious tracks no real garment received a wrong verdict. A new identity that appears on a pile is never armed, so it cannot repeat a verdict, and a spurious track can add a record but cannot change a real one. The arming rule therefore turns two of the most common tracking errors into, at worst, a miss or an extra record, both of which leave the existing records intact.

6.1.3 Effect of a confirmation rule

The current logic logs a verdict on the very first frame in which an armed track is in the exit zone. Experiment S7 asked whether requiring k consecutive observations in the same exit zone would help. Figure 6.6 and Table 6.4 show the counting error for each value of k under five conditions.

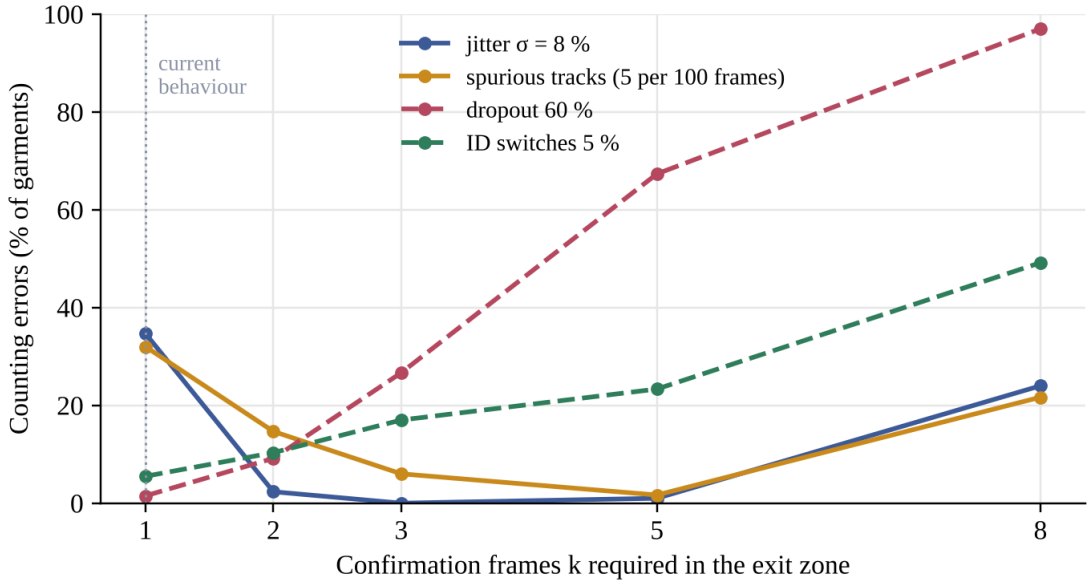


Figure 6.6: Counting error as a function of the number of confirmation frames k

Table 6.4: Counting error (%) for different confirmation lengths k (600 garments each)

Condition	k = 1	k = 2	k = 3	k = 5	k = 8
Jitter 6 %	13.8	0.3	0.0	0.5	23.7
Jitter 8 %	34.7	2.3	0.0	1.0	24.0
Spurious 5 %	32.0	14.7	6.0	1.7	21.7
Dropout 60 %	1.5	9.2	26.7	67.3	97.0
ID switch 5 %	5.5	10.3	17.0	23.3	49.2

A confirmation of three frames removed every jitter error, at both 6 % and 8 % jitter, and cut the extra records from spurious tracks from 32.0 % to 6.0 % of garments. But it raised misses when detections drop out, from 1.5 % to 26.7 % at 60 % dropout, and when identities switch, from 5.5 % to 17.0 % at 5 % switches, because a garment that stays in the exit zone for only a few frames may not supply enough consecutive observations. Long confirmations were harmful in every condition. The best value therefore depends on which errors dominate at a particular station, and on how long a garment remains visible on the pile after it is placed, which in this simulation was only two to eight frames.

6.2 Product and technical results

Software verification. All 51 backend tests and all three frontend rework tests passed, with 94 % statement coverage of the service (Section 5.4). The suite verified every functional requirement in Table 4.1 that can be checked without a camera, and found the defects listed in Table 5.5.

Repeatability of the full pipeline. The seed database holds three sessions from replays of the same workflow video on 28 August 2026, summarised in Table 6.5 and Figure 6.7. The first session was ended after one verdict. The second and third ran to the end of the video and produced the same five verdicts, in the same order, under the same tracker identifiers: Rework for identifiers 14, 19, 45 and 51 and Good for identifier 35. The times of the verdicts after the start of the session agreed within two seconds, from 50 and 51 seconds for the first verdict to 170 and 171 seconds for the last.

Table 6.5: Sessions recorded from replays of the workflow video

Session (SESSION_ ...)	Verdicts	Good	Rework	Pass rate	Delay at 2.0 min	First to last verdict
20260828_061454	1	0	1	0.0 %	2.0 min	06:15:46
20260828_061611	5	1	4	20.0 %	8.0 min	06:17:01 to 06:19:01
20260828_062230	5	1	4	20.0 %	8.0 min	06:23:21 to 06:25:21

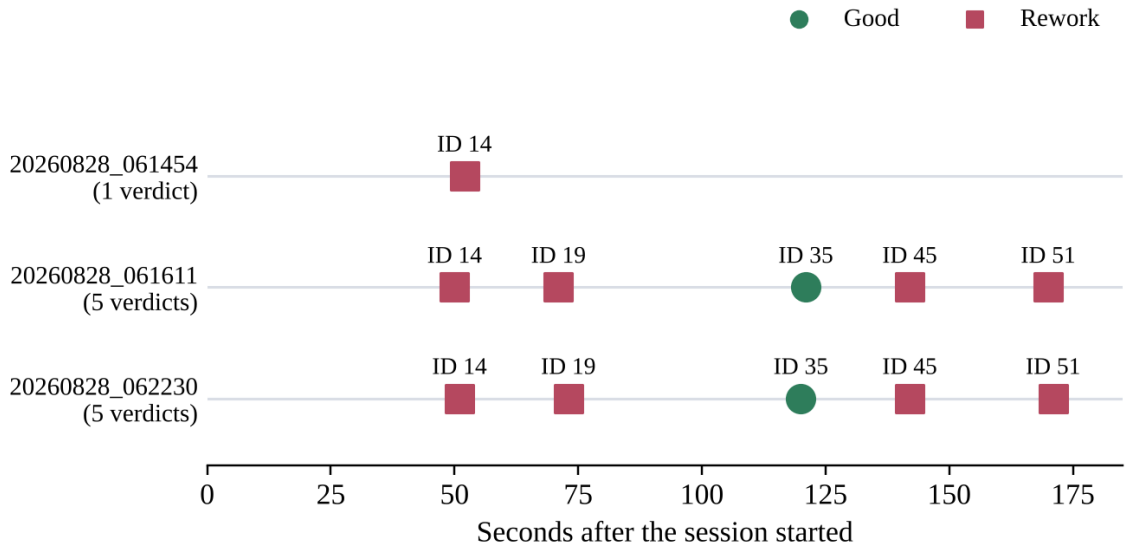


Figure 6.7: Verdicts of the three stored sessions over time

Two further observations come from these sessions. First, the whole pipeline, from frame decoding through detection, tracking and the verdict rule to the database, is deterministic for a given video, which makes workflow videos a sound basis for regression testing. Second, the highest tracker identifier reached 51 while only five garments were counted, so many more tracks were started than garments were judged. Most of these short tracks never satisfied the arming rule, which is the behaviour the design intends, but they are also the raw material of the spurious records studied in Section 6.1.2. Because no independent annotation of the video was kept, these sessions demonstrate repeatability, not accuracy.

Session analytics. The pass rate and delay in Table 6.5 follow Equation 3.3. For the complete sessions, one Good and four Rework verdicts give a pass rate of 20.0 %, shown in red in the session history, and with the default fix time of 2.0 minutes an estimated delay of 8.0 minutes. The figures were checked by hand and agree with what the interface displays.

6.3 User or stakeholder validation

No structured usability study with factory inspectors or supervisors was carried out within the project period; this is future work. Table 6.6 instead maps the weaknesses of manual recording at a checking table, as described in the quality literature [5], [12], to the features of the component that address them.

Table 6.6: Weaknesses of manual rework recording and the corresponding features

Weakness of manual recording	Feature of the component
Recording competes with inspection for the inspector's attention	Verdict read from the movement the inspector already makes
Totals reach the supervisor at the end of the shift	Counts and pass rate updated every second
No time record of individual decisions	Each verdict stored with session and time
Rework is reported as a count, not as lost time	Estimated delay from an average fix time
Hard to compare shifts or batches	Session history with pass rate and CSV export

6.4 Integration results

Within the platform, the Rework Counter card starts its service, shows the shared startup screen until the health check succeeds, and opens the Live production view. Sessions were run in the integrated application with a webcam and with workflow

videos. Requests outside the allow list are refused by the main process before they reach the service, the preview survives indefinitely now that the stream has no timeout, the preview reconnects after a restart, and leaving the feature releases the camera for the next one. The three frontend tests and the platform's layout tests confirm the integration rules in the source. The stored sessions in Table 6.5 show the complete chain, from input through the service and database to the session history, operating end to end.

6.5 Summary of findings

- The single class YOLOv8n detector reached mAP@0.5 0.938, precision 0.895 and recall 0.869 on its validation split; box tightness, not detection, accounts for most of the remaining error (RQ1: yes).
- On clean tracks the zone rule recorded all 600 simulated garments correctly and exactly once, and two replays of a workflow video gave identical verdicts (RQ2: yes).
- The logic tolerates missed detections well but is sensitive to centre jitter near the lines, to identity switches during the exit movement and to spurious short tracks; jitter causes wrong verdicts, switches cause misses, and spurious tracks cause extra records (RQ3).
- A three frame confirmation removes jitter errors and most spurious records in simulation, but increases misses under heavy dropout or frequent switches (RQ4: yes, with a trade off).
- The component runs as an isolated, allow listed service of the platform, verified by 54 passing tests with 94 % coverage of the service (RQ5: yes).

7 DISCUSSION

7.1 Interpretation of findings

The most important result is where the risk in this component lies. The detector is good enough: with one class and a forgiving use of its output, its validation scores leave little to gain. The counting code is correct on clean input. What decides whether the counts are right in practice is the interaction between the verdict rule and the imperfections of tracking near the two lines, exactly as the working hypothesis predicted.

The simulation makes the nature of that interaction clear. Each imperfection produces its own kind of error. Jitter produces wrong verdicts, because the rule acts on the first frame in which an armed centre is past a line, and a hand that tilts the garment during inspection can push the centre past a line for a moment. Identity switches produce misses, because a new identity that first appears in the exit zone is, by design, not armed. Spurious tracks produce extra records. Missed detections are largely harmless. These are different failures with different consequences: a wrong verdict moves a garment from one count to the other, leaving the total right but the pass rate wrong; a miss lowers the total; an extra record raises it. A supervisor would be misled in different ways by each, and a single accuracy figure would hide the difference.

The arming rule deserves credit for what did not happen. There were no double counts under identity switches and no corrupted real verdicts under spurious tracks. Requiring a track to be seen in the middle zone before it can count is a simple idea, but it converts the two most common tracking failures into the least harmful errors. A plain line crossing counter would have counted garments lying on the piles each time the tracker gave them a new identity.

The confirmation experiment shows that robustness is a trade off rather than a setting to maximise. A short confirmation helps against jitter and spurious tracks and hurts when tracks are short or broken. In the simulation garments stayed in the exit zone for only two to eight frames, which is pessimistic; on a real table a garment placed on a pile usually remains visible, so the cost of confirmation would be smaller. The right

value can only be chosen from measurements at a real station, which is why Chapter 9 places a field trial first among the future directions.

Finally, finding F2 changes how the component should be understood. Because counting happens inside the stream generator, the preview is not just a view of the counter; it is what makes the counter run. Two of the three integration defects fixed during this work, the ten second stream timeout and the unmounted live view, were in fact counting defects that looked like display defects. Separating the frame loop from the stream would remove this whole class of problem.

7.2 Comparison with literature baselines and existing solutions

The detector's behaviour fits the general picture for nano scale YOLO models fine tuned from COCO [3], [8]: fast convergence on the easier mAP@0.5 and slow, continuing gains on the stricter metric. Direct comparison with tracking benchmarks such as MOT16 [29] would mislead, because those measure identity preservation for many pedestrians, whereas this task has one or two objects at a time and cares about a single event per object. The finding that identity switches are the most damaging tracker error agrees with the tracking literature [4], [15], [28], but here its effect is specific: a switch matters only if it happens in the short interval between leaving the middle zone and being counted.

Compared with line crossing counters [13], [9], the component adds the inspection zone, the arming condition and the mapping of direction to a quality verdict. Compared with automated defect inspection [10], [11], it solves a narrower problem, recording decisions rather than making them, but needs no defect data and no controlled imaging, and it leaves the inspector's judgement intact. Against the recording methods in Table 2.2, it is the only approach that is real time and adds no action for the inspector, at the cost of being unable to identify individual garments for traceability. The observation that the reliability of the system comes from the code around the model echoes Sculley et al. [30].

7.3 Research contribution

First, the use of the direction of a human inspector's handling movement as the label of a quality decision, implemented as a two line zone rule with an arming condition

over tracker output. Second, an evaluation method that isolates the event logic from the detector and tracker by driving the shipped counting code with synthetic trajectories whose verdicts are known, together with a characterisation of how jitter, dropout, identity switches and spurious tracks each translate into a different kind of counting error. Third, evidence that a short confirmation rule trades jitter and spurious errors against misses, and that its best length depends on how long garments remain visible after placement.

7.4 Product contribution

The component is a working, integrated feature of the Garment Intelligence platform that counts accepted and rework garments in real time from a webcam or a workflow video, stores every verdict per session, and presents counts, pass rate, estimated delay, charts, history and CSV export. It starts from a workstation card, runs as an isolated service on a private port, accepts only allow listed requests, shows what it sees, and releases the camera when the operator leaves. It is covered by 54 passing tests, which it did not have before this work.

7.5 Limitations and threats to validity

- **No field accuracy.** No session at a working line was compared with an independent manual count, so the field rates of wrong verdicts, misses and extra records are unknown.
- **Synthetic trajectories.** The simulation uses independent Gaussian jitter, independent dropouts and a simple model of spurious tracks. Real jitter is correlated over frames and real spurious tracks have structure, so the numbers indicate sensitivities and mechanisms, not field error rates.
- **Stubbed detector and tracker in tests.** The tests and simulation replace the model and ByteTrack, so they verify the code around them, not their behaviour; the replays cover the real chain but without ground truth.
- **Dataset documentation.** The detector's metrics come from its own validation split; the composition and capture conditions of the dataset are reported only in summary, and no separate test split result is reported.

- **Fixed station layout.** The direction to verdict mapping and the single camera geometry assume a table with the good pile above and the rework pile below the inspection position.
- **Engineering limits.** Counting depends on the stream being read (F2); session names have one second resolution (F5); times lack dates (F6); track states grow within a session (F7); line settings are not validated by the service (F8); the fix time is neither shared between views nor stored (F9).

Taken together, these limitations mean that the thesis supports strong claims about the correctness of the counting code, the behaviour of the verdict rule under controlled imperfections, the repeatability of the pipeline and the quality of the detector on its validation data, but only cautious claims about counting accuracy at a working line.

7.6 Implications

For apparel manufacturers, the work suggests that rework can be counted automatically at an ordinary checking table with a webcam and a small model, without changing the inspector's routine, provided the station is laid out so that the inspection position and the two piles are clearly separated in the camera's view. Its value is in timeliness: a pass rate and an estimated delay that a supervisor can act on during the shift. For engineers building counters on top of detectors and trackers, the work shows that the event rule deserves its own tests and its own robustness study, because benchmark metrics for detection and tracking do not measure it, and that each kind of tracking error should be traced to the kind of counting error it causes. For this component, the next gains will come from separating counting from the preview, adding a short confirmation with hysteresis around the lines, and calibrating both against a real station.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The component turns an ordinary USB camera and an existing factory computer into a live record of quality decisions: instead of learning at the end of the shift that rework rose in the morning, a supervisor sees the pass rate fall within minutes and can go to the sewing line while the cause is still there. Table 8.1 shows who benefits. The buyers would be apparel manufacturers, especially small and medium plants that cannot justify tagging every garment or installing automated inspection.

Table 8.1: Benefits of the component for each user group

User group	Benefit
QC inspector	No tally sheet and no button to press; attention stays on the garment
Line supervisor	Live Good and Rework counts, pass rate and estimated delay during the shift
Production planner	Session history and CSV export to revise delivery estimates
Quality manager	Timed record of decisions per session for trend analysis and audits

8.2 Market and competitive position

The apparel sector is a major Sri Lankan export industry [1] with many checking tables in every factory, and the same need exists across the region. The alternatives are paper tallies, which cost nothing but are late and add work; tablet entry, which is timely but adds a step to every garment; tag based tracking, which gives traceability at the cost of tags, readers and scanning; and automated defect inspection, which is powerful but expensive and specific to fabrics and defects. The component's differentiators are that it adds no work for the inspector, needs only hardware the factory already has, keeps the human decision as the record, and comes as part of a platform that also covers sewing, packing and sizing.

8.3 Business or sustainability model

A suitable model is a modest installation fee covering camera mounting, line calibration and supervisor training, plus a subscription per station covering updates and support. Because the software runs on the customer's own computer, each additional station costs little to serve, so pricing per station scales with the size of the customer, and the Rework Counter can be sold alone or with the other platform

features. Table 8.2 outlines the cost structure of one station; actual prices would depend on local market research outside the scope of this thesis.

Table 8.2: Indicative cost structure for one quality checking station

Item	Nature of cost
Camera and overhead mount	One time, low cost consumer hardware
Workstation	Usually an existing PC
Installation and calibration	One time service per station
Software subscription	Recurring, per station
Model retraining for new garment types	Included in subscription or charged per project

A pilot on one line, run alongside the existing manual count for several weeks, would supply the independent ground truth this thesis lacks, show how often each kind of error occurs at a real station, and provide a case study for further sales.

8.4 Scalability intellectual property and compliance

Scaling to a whole factory would mean one service per checking table sending verdicts to a central database instead of a local file; because all SQL sits in two small functions, this change is contained. Authentication, a production server process and date stamped records would be added at that stage. The Ultralytics framework is released under the AGPL 3.0 licence, with an enterprise licence available for commercial products [3]; a commercial release would either comply with the AGPL or obtain that licence, and the other dependencies would be reviewed in the same way. Cameras in a workplace raise surveillance concerns. Looking down at the table, keeping no video, storing no images, counting per session rather than per person, and leaving the decision to the inspector address these, and a deployment would also need worker consultation, clear signage and compliance with Sri Lanka's Personal Data Protection Act and buyers' social compliance audits.

8.5 Contribution to the Sustainable Development Goals

The component supports several Sustainable Development Goals [36]. For SDG 12 (responsible consumption and production), earlier visibility of rework helps to stop a defect at its source, reducing repeated labour, wasted thread and fabric, and garments that are finally rejected. For SDG 9 (industry, innovation and infrastructure), it brings affordable digital quality tools to small and medium manufacturers. For SDG 8 (decent

work), it removes a repetitive recording task from inspectors and, by keeping their judgement as the record, supports rather than replaces skilled work. Its modest hardware needs also keep its energy use and cost low.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis replaced the delayed manual recording of accept and reject decisions at an apparel quality checking table with a camera based component that reads each decision from the direction in which the inspector moves the garment. A single class YOLOv8 nano detector reached an mAP@0.5 of 0.938 on its validation split; ByteTrack keeps each garment's identity; and a two line zone rule with an arming condition records a Good or Rework verdict once for every garment that is inspected in the middle of the view and then placed on a pile. The verdicts are stored per session and shown as live counts, pass rate, estimated delay and history, inside an isolated feature of the Garment Intelligence platform. The central lesson is that the reliability of such a counter is decided by its event logic. On clean tracks the logic is exact; under realistic imperfections each tracking error becomes a specific counting error, and the arming rule limits the worst of them. All 54 tests passed, and two replays of a workflow video produced identical results.

9.2 Achievement of objectives

Table 9.1: Achievement of the research objectives

Objective	Summary	Evidence
O1	Lightweight single class detector	mAP@0.5 0.938, precision 0.895, recall 0.869 (6.1.1)
O2	Verdict from movement direction	Zone rule and state machine (4.4.3); 600 of 600 correct on clean tracks (6.1.2)
O3	Persistence and quality information	Session store, counts, pass rate, delay, history, export (4.4.4, 6.2)
O4	Isolated platform feature	Separate service, allow listed IPC, stream protocol (5.3, 6.4)
O5	Verification	54 of 54 tests passed, 94 % coverage; repeatable replays (5.4, 6.2)
O6	Robustness characterised	Seven simulation experiments; confirmation trade off (6.1.2, 6.1.3)

All six objectives were met (Table 9.1), with the qualification that counting accuracy at a working line was not measured.

9.3 Individual research and product contributions

The research contribution is the idea of treating the inspector's handling movement as the label of a quality decision, an evaluation method that isolates the event logic by driving the shipped code with synthetic trajectories, and the resulting map from tracking imperfections to counting errors, including the trade off of a confirmation rule. The product contribution is the working component: detector, service, verdict logic, session store, views, platform integration and test suite.

Beyond its specific results, the work leaves a stable base for further development. The verdict record, the endpoint contract and the test harness allow the improvements below to be made and checked without changing the rest of the platform, and the simulation scripts can be rerun after each change to show whether robustness has improved.

9.4 Future directions

- Run a multi week trial at a working checking table against an independent manual count, and annotate workflow videos, to measure the field rates of wrong verdicts, misses and extra records.
- Move the frame loop into a background worker so that counting no longer depends on the preview, and serve the stream from the latest annotated frame.
- Add a short confirmation with hysteresis around each line, with its length chosen from the field measurements, and remove the unreachable missing frame branch.
- Re-arm or link a new track identity that appears just past a line shortly after a track was lost in the middle zone, to recover misses caused by identity switches.
- Store full date and time stamps, give sessions unique identifiers, prune finished tracks, validate line settings in the service, and store the fix time with the session.
- Monitor the pass rate with a p chart [18] so that an unusual rise in rework triggers an alert, and record a reason code for rework when a simple input is acceptable to inspectors.

Automating quality records at the end of an apparel line does not require a machine to judge the garments. It requires a clear view of the decision the inspector has already made, and careful logic that turns that view into a record that can be trusted.

REFERENCES

[1] Sri Lanka Export Development Board, "Apparel and textile industry in Sri Lanka," [Online]. Available: https://www.srilankabusiness.com/apparel/

[2] H. Lasi, P. Fettke, H.-G. Kemper, T. Feld, and M. Hoffmann, "Industry 4.0," *Business & Information Systems Engineering*, vol. 6, no. 4, pp. 239–242, 2014.

[3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[4] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. European Conf. Computer Vision (ECCV)*, 2022, pp. 1–21.

[5] C. N. Keist, "Quality control and quality assurance in the apparel industry," in *Garment Manufacturing Technology*, R. Nayak and R. Padhye, Eds. Cambridge, U.K.: Woodhead Publishing, 2015, pp. 447–466.

[6] J. M. Juran and A. B. Godfrey, *Juran's Quality Handbook*, 5th ed. New York, NY, USA: McGraw-Hill, 1999.

[7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[8] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023.

[9] Roboflow, "Supervision: Reusable computer vision tools," GitHub repository, 2023. [Online]. Available: https://github.com/roboflow/supervision

[10] A. Kumar, "Computer-vision-based fabric defect detection: A survey," *IEEE Trans. Industrial Electronics*, vol. 55, no. 1, pp. 348–363, 2008.

[11] H. Y. T. Ngan, G. K. H. Pang, and N. H. C. Yung, "Automated fabric defect detection—A review," *Image and Vision Computing*, vol. 29, no. 7, pp. 442–458, 2011.

[12] J. E. See, "Visual inspection: A review of the literature," Sandia National Laboratories, Albuquerque, NM, USA, Rep. SAND2012-8590, 2012.

[13] Z. Ma and A. B. Chan, "Crossing the line: Crowd counting by integer programming with local features," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2013, pp. 2539–2546.

[14] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2016, pp. 3464–3468.

[15] K. Bernardin and R. Stiefelhagen, "Evaluating multiple object tracking performance: The CLEAR MOT metrics," *EURASIP J. Image and Video Processing*, vol. 2008, Art. no. 246309, 2008.

[16] A. V. Feigenbaum, *Total Quality Control*, 3rd ed. New York, NY, USA: McGraw-Hill, 1983.

[17] T. Ohno, *Toyota Production System: Beyond Large-Scale Production*. Cambridge, MA, USA: Productivity Press, 1988.

[18] D. C. Montgomery, *Introduction to Statistical Quality Control*, 7th ed. Hoboken, NJ, USA: Wiley, 2013.

[19] S. Nakajima, *Introduction to TPM: Total Productive Maintenance*. Cambridge, MA, USA: Productivity Press, 1988.

[20] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 91–99.

[21] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.

[22] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The PASCAL visual object classes (VOC) challenge," *Int. J. Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.

[23] R. Padilla, S. L. Netto, and E. A. B. da Silva, "A survey on performance metrics for object-detection algorithms," in *Proc. Int. Conf. Systems, Signals and Image Processing (IWSSIP)*, 2020, pp. 237–242.

[24] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Trans. ASME, J. Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[25] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.

[26] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2017, pp. 3645–3649.

[27] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, "BoT-SORT: Robust associations multi-pedestrian tracking," arXiv:2206.14651, 2022.

[28] J. Luiten, A. Ošep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe, "HOTA: A higher order metric for evaluating multi-object tracking," *Int. J. Computer Vision*, vol. 129, no. 2, pp. 548–578, 2021.

[29] A. Milan, L. Leal-Taixé, I. Reid, S. Roth, and K. Schindler, "MOT16: A benchmark for multi-object tracking," arXiv:1603.00831, 2016.

[30] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 2503–2511.

[31] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[32] Roboflow, "Roboflow: Computer vision developer platform," [Online]. Available: https://roboflow.com

[33] S. Ramírez, "FastAPI," FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/

[34] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.

[35] Electron, "Context isolation," Electron Documentation. [Online]. Available: https://www.electronjs.org/docs/latest/tutorial/context-isolation

[36] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, Oct. 2015.

GLOSSARY

Term	Definition
Arming	The step in which a track becomes eligible for a verdict, because its centre has been seen in the middle zone inside the centre band.
Centre band	The horizontal range of the frame, 10 % to 90 % of the width by default, outside which every box centre is treated as being in the middle zone.
Confirmation frames	In the simulated refinement, the number of consecutive observations in the same exit zone required before a verdict is logged.
Estimated delay	The number of Rework verdicts multiplied by the average fix time entered by the user.
Exit zone	The UP zone above the top line or the DOWN zone below the bottom line.
First pass yield	The share of garments accepted at their first inspection; reported here as the pass rate.
Identity switch	A tracking error in which the same physical object receives a new tracker identifier.
Pass rate	Good verdicts divided by all verdicts in a session, as a percentage.
Rework	A garment returned for repair after inspection, and the repeated work this causes.
Session	A period of counting started with Start New Session and ended with End Session, the end of a video or leaving the feature.
Sidecar	A separate service process started and supervised by the desktop application for one feature.
Spurious track	A short lived track created by something other than a garment being inspected, such as a hand, a tool or a reflection.
Verdict	The Good or Rework decision recorded for one garment.
Workflow video	A recorded video of inspection work that can be used as the input of a session in place of a webcam.

APPENDICES

The appendices provide supplementary material referred to in the main text.

Appendix A Complete list of automated test cases

Table A.1 lists every test described in Section 5.4 with the behaviour it verifies. All 54 tests passed when the suites were executed, so the outcome of every case listed is Pass. Tests whose names begin with Known limit assert the current behaviour of a documented limitation.

Table A.1: Automated test cases and outcomes

ID	Behaviour verified	ID	Behaviour verified
IT_01	Health reports the feature and its paths	UT_C07	Exit outside the centre band is ignored
IT_02	Metrics before any session are zero	UT_C08	MID visit outside the band does not arm
IT_03	Unsupported source type is rejected	UT_C09	Simultaneous garments counted independently
IT_04	Non numeric camera index is rejected	UT_C10	Top boundary is strict
IT_05	Missing workflow video is rejected	UT_C11	Bottom boundary is strict
IT_06	Camera that cannot open reports an error	UT_C12	Line positions follow settings
IT_07	Start returns a time stamped session name	UT_C13	Track lost in MID is not counted
IT_08	Second start while running is refused	UT_C14	Verdict logged on first frame in exit zone
IT_09	Video file source is accepted	UT_C15	Missing frame branch adds no second record
IT_10	Live settings update the running session	UT_C16	ID switch in MID still counts once
IT_11	Stop releases the camera	UT_C17	Known limit: new ID after count can recount
IT_12	Stream is refused when not running	UT_C18	Known limit: one frame jitter across a line counts
IT_13	Stream delivers one JPEG part per frame	UT_C19	Frames without tracker IDs are handled
IT_14	Malformed body is rejected by validation	UT_C20	History holds cumulative counts per frame
IT_15	Metrics count only the current session	UT_C21	History capped at 500, API returns 100
IT_16	Session history gives counts and pass rate	UT_C22	Confidence threshold reaches the detector
IT_17	Sessions are listed most recent first	UT_C23	One JPEG part per processed frame
IT_18	Log is newest first and capped at 50	UT_D01	Schema created on import
IT_19	New session resets live counters	UT_D02	Initialisation is idempotent
IT_20	Model is loaded once across sessions	UT_D03	Data directory comes from the environment
IT_21	End of video marks the session stopped	UT_D04	Timestamp stored as clock time
UT_C01	MID then top is counted Good	UT_D05	Values bound as parameters
UT_C02	MID then bottom is counted Rework	UT_D06	Known limit: midnight session times misordered
UT_C03	Track never in MID is not counted	UT_D07	Known limit: same second sessions share a name

ID	Behaviour verified	ID	Behaviour verified
UT_C04	Garment kept in MID is not counted	FE_01	Stream scheme allowed by the renderer CSP
UT_C05	Each track is counted only once	FE_02	Stream proxy has no ten second abort
UT_C06	First verdict is final	FE_03	Workflow video picker and reconnecting preview

Appendix B Simulation details and complete results

Each condition of the simulation study used three episodes of 200 garments generated with fixed seeds. Garments appear at a horizontal position between 35 % and 65 % of the width and a vertical position between 42 % and 58 % of the height, stay for 30 to 90 frames, move over 5 to 12 frames to a target between 5 % and 20 % (good) or 80 % and 95 % (rework) of the height, remain for 2 to 8 frames and then disappear, followed by 5 to 20 empty frames. Jitter is applied to every frame in the inspection phase and at half strength during the movement and on the pile. Table B.1 gives the complete results of experiments S1 to S6.

Table B.1: Complete results of the simulation study (600 garments per row)

Exp.	Condition	Correct	Wrong	Missed	Double	Extra
S1	Clean	600	0	0	0	0
S2	Jitter σ 0 %	600	0	0	0	0
S2	Jitter σ 2 %	600	0	0	0	0
S2	Jitter σ 4 %	594	6	0	0	0
S2	Jitter σ 6 %	513	87	0	0	0
S2	Jitter σ 8 %	384	216	0	0	0
S2	Jitter σ 10 %	343	257	0	0	0
S2	Jitter σ 12 %	313	287	0	0	0
S4	Dropout 0 %	600	0	0	0	0
S4	Dropout 20 %	600	0	0	0	0
S4	Dropout 40 %	599	0	1	0	0
S4	Dropout 60 %	593	0	7	0	0
S4	Dropout 80 %	522	0	78	0	0
S4	Dropout 90 %	384	0	216	0	0
S5	ID switch 0 %	600	0	0	0	0
S5	ID switch 1 %	595	0	5	0	0
S5	ID switch 2 %	594	0	6	0	0
S5	ID switch 5 %	572	0	28	0	0
S5	ID switch 10 %	541	0	59	0	0
S6	Spurious 0 %	600	0	0	0	0
S6	Spurious 1 %	600	0	0	0	34
S6	Spurious 2 %	600	0	0	0	75
S6	Spurious 5 %	600	0	0	0	180
S6	Spurious 10 %	600	0	0	0	364
S3	Lines 40 / 60 %, σ 4 %	366	234	0	0	0

Exp.	Condition	Correct	Wrong	Missed	Double	Extra
S3	Lines 40 / 60 %, σ 8 %	300	300	0	0	0
S3	Lines 35 / 65 %, σ 4 %	518	82	0	0	0
S3	Lines 35 / 65 %, σ 8 %	309	291	0	0	0
S3	Lines 30 / 70 %, σ 4 %	594	6	0	0	0
S3	Lines 30 / 70 %, σ 8 %	384	216	0	0	0
S3	Lines 25 / 75 %, σ 4 %	600	0	0	0	0
S3	Lines 25 / 75 %, σ 8 %	519	81	0	0	0
S3	Lines 20 / 80 %, σ 4 %	599	0	1	0	0
S3	Lines 20 / 80 %, σ 8 %	583	17	0	0	0